

```
1 // ===== FILE: src/main.ts =====
2 /**
3  * 村口大战外星人 · 游戏入口
4  *
5  * 先出标准 Loading（插画 + 进度条 + 健康游戏忠告），资源就绪再落村子。
6  */
7 import '@core/pixiUnsafeEvalPatch';
8 import { analytics, initAnalytics, setAnalyticsUserId } from '@analytics';
9 import { MAIN_PRELOAD_IMAGES, VILLAGE_HOME_SHELL } from '@config/assetPreload';
10 import { warmupCdnAssets } from '@config/CdnWarmup';
11 import { SAVE_KEY } from '@config/CloudConfig';
12 import { BASE_GAME_KEY } from '@config/gameKeyScope';
13 import { CdnAssetService } from '@core/CdnAssetService';
14 import { BackendService } from '@core/BackendService';
15 import { CloudSyncManager } from '@core/CloudSyncManager';
16 import { EventBus } from '@core/EventBus';
17 import { ensureAssets } from '@core/ensureAssets';
18 import { Game } from '@core/Game';
19 import { PersistService } from '@core/PersistService';
20 import { SceneManager } from '@core/SceneManager';
21 import { Platform } from '@core/PlatformService';
22 import { OverlayManager } from '@core/OverlayManager';
23 import { BgmPlayer } from '@core/BgmPlayer';
24 import { GMManager } from '@core/GMManager';
25 import { BattleScene } from '@scenes/BattleScene';
26 import { RoadScene } from '@scenes/RoadScene';
27 import { VillageScene } from '@scenes/VillageScene';
28 import { LoadingScreenOverlay } from '@ui/LoadingScreenOverlay';
29 import { GMEntryButton } from '@ui/GMEntryButton';
30 import { GMPanel } from '@ui/GMPanel';
31 declare const GameGlobal: any;
32 const MIN_LOADING_MS = 900;
33 function sleep(ms: number): Promise<void> {
34   return new Promise((resolve) => setTimeout(resolve, ms));
35 }
36 initAnalytics();
37 CloudSyncManager.prewarm();
38 PersistService.subscribeCloudImport((info) => {
39   if (!info.changedKeys.includes(SAVE_KEY)) return;
40   EventBus.emit('home:refresh');
41 });
42 if (typeof GameGlobal !== 'undefined') {
43   const prevError = GameGlobal.onError;
44   const prevReject = GameGlobal.onUnhandledRejection;
45   GameGlobal.onError = (msg: string) => {
46     console.error('[GlobalError]', msg);
47     try { prevError?.(msg); } catch { /* */ }
48     analytics.trackAppError(msg, { source: 'GameGlobal.onError' });
49   };
50   GameGlobal.onUnhandledRejection = (ev: { reason?: unknown }) => {
```

```
51 console.error('[UnhandledRejection]', ev?.reason || ev);
52 try { prevReject?.(ev); } catch { /* */ }
53 analytics.trackAppError(ev?.reason || ev, { source: 'unhandledRejection' });
54 };
55 }
56 async function resolveUserId(): Promise<string> {
57   const startupSync = await CloudSyncManager.awaitStartupSync();
58   console.log(`[main] 云同步启动结果: ${startupSync.status}, reason=${startupSync.reason}`);
59   let userId = CloudSyncManager.userId;
60   if (!userId && BackendService.available) {
61     try {
62       await BackendService.ensureToken();
63       userId = BackendService.userId;
64     } catch (error) {
65       console.warn(`[main] 启动后补登失败（不挡玩）`, error);
66     }
67   }
68   return userId;
69 }
70 async function main(): Promise<void> {
71   const canvas = GameGlobal?.canvas ?? null;
72   if (!canvas) {
73     console.error(`[main] 找不到 canvas`);
74     return;
75   }
76   Game.init(canvas);
77   if (!Game.app?.renderer) {
78     console.error(`[main] 渲染器初始化失败`);
79     return;
80   }
81   console.log(`[main] 启动 平台=${Platform.name} gameKey=${BASE_GAME_KEY} 版本=${__APP_VERSION__}`);
82   const userIdP = resolveUserId();
83   Game.stage.sortableChildren = true;
84   const loading = new LoadingScreenOverlay();
85   Game.stage.addChild(loading);
86   Game.syncFrameToScreen();
87   const shownAt = Date.now();
88   await ensureAssets(MAIN_PRELOAD_IMAGES);
89   loading.applySplashTexture();
90   loading.applyTitleTexture();
91   loading.setProgress(0.18);
92   Game.syncFrameToScreen();
93   CdnAssetService.prepareLocalCache();
94   warmupCdnAssets();
95   await CdnAssetService.fetchManifest().catch(() => false);
96   loading.setProgress(0.28);
97   // 村口壳在分包里，出插画后再等，避免主界面空壳。立绘 / 战斗仍后台 CDN。
98   await ensureAssets(VILLAGE_HOME_SHELL, (loaded, total) => {
99     loading.setProgress(0.28 + 0.58 * (total > 0 ? loaded / total : 1));
100   }).catch((e) => {
```

```
101   console.warn('[main] 村口壳预热失败', e);
102   });
103   loading.setProgress(0.86);
104   const userId = await userIdP;
105   loading.setProgress(0.92);
106   SceneManager.register(new VillageScene());
107   SceneManager.register(new RoadScene());
108   SceneManager.register(new BattleScene());
109   SceneManager.switchTo('village');
110   if (GMMManager.isRuntimeAllowed) {
111     OverlayManager.container.addChild(new GMPanel());
112     OverlayManager.container.addChild(new GMEntryButton());
113   }
114   await Game.warmScenePresent();
115   loading.setProgress(1);
116   Game.syncFrameToScreen();
117   const wait = MIN_LOADING_MS - (Date.now() - shownAt);
118   if (wait > 0) await sleep(wait);
119   Game.stage.removeChild(loading);
120   loading.destroy({ children: true });
121   Game.syncFrameToScreen();
122   // game.js 的启动超时诊断靠这个标记判断是否真的画出来了
123   if (typeof GameGlobal !== 'undefined') GameGlobal.__gameRendered = true;
124   if (userId) {
125     setAnalyticsUserId(userId);
126   } else {
127     console.warn('[main] 未拿到登录 userId, 经分仅以 anonymous_id 上报);
128   }
129   analytics.trackSessionStart({
130     entry: 'main_boot',
131     with_user_id: !!userId,
132     cloud_sync_ready: CloudSyncManager.ready,
133   });
134   let lastHideAt = 0;
135   Platform.onHide() => {
136     BgmPlayer.pause();
137     void CloudSyncManager.flushNow('app-hide');
138     analytics.trackSessionEnd('app-hide');
139     lastHideAt = Date.now();
140   };
141   Platform.onShow() => {
142     BgmPlayer.resume();
143     if (lastHideAt > 0) {
144       analytics.trackAppShow({
145         background_ms: Date.now() - lastHideAt,
146       });
147     }
148   };
149 }
150 main().catch((e) => {
```

```
151 console.error('[main] 启动失败:', e);
152 });
153 // ===== FILE: src/core/PlatformService.ts =====
154 /**
155  * 平台服务抽象层 - 统一封装微信/抖音双平台 API
156  *
157  * 业务侧跨平台 SDK 入口: 所有 wx/tt 差异 (存储、登录、分享、生命周期等)
158  * 都必须走 Platform, 禁止在业务里写 typeof wx / typeof tt。
159  *
160  * 宿主识别: 抖音注入 tt (可同时存在 wx 兼容层); 微信仅 wx。
161  * 检测到哪个宿主, 就只绑定该宿主原生 API (抖音→tt, 微信→wx), 互不倒用。
162  */
163 declare const wx: any;
164 declare const tt: any;
165 export type PlatformName = 'wechat' | 'douyin' | 'unknown';
166 export type BackendPlatformCode = 'wx' | 'dy' | 'anon';
167 /** 检测当前小游戏宿主 (单一真源, 与 runtime/runtime.js 逻辑一致) */
168 export function detectMinigamePlatform(): PlatformName {
169   if (typeof tt !== 'undefined') return 'douyin';
170   if (typeof wx !== 'undefined') return 'wechat';
171   return 'unknown';
172 }
173 /** 指定宿主的原生 API: 抖音仅 tt, 微信仅 wx */
174 export function getNativePlatformApi(platform: PlatformName = detectMinigamePlatform()): any {
175   if (platform === 'douyin') return typeof tt !== 'undefined' ? tt : null;
176   if (platform === 'wechat') return typeof wx !== 'undefined' ? wx : null;
177   return null;
178 }
179 /** @deprecated 请用 detectMinigamePlatform + getNativePlatformApi */
180 export function resolveMinigameRuntime(): { name: PlatformName; api: any } {
181   const name = detectMinigamePlatform();
182   return { name, api: getNativePlatformApi(name) };
183 }
184 export function toBackendPlatformCode(name: PlatformName): BackendPlatformCode {
185   if (name === 'douyin') return 'dy';
186   if (name === 'wechat') return 'wx';
187   return 'anon';
188 }
189 class PlatformServiceClass {
190   /** 当前平台名 */
191   readonly name: PlatformName;
192   /** 底层平台 API 对象 (wx / tt / null) */
193   private _api: any;
194   /** adUnitId → 激励视频实例 (宿主本身也是单例, 这里避免重复注册回调) */
195   private _adCache = new Map<string, any>();
196   /** 当前在播广告的 Promise resolver, null = 无广告在播 */
197   private _adResolve: ((ok: boolean) => void) | null = null;
198   constructor() {
199     this.name = detectMinigamePlatform();
200     this._api = getNativePlatformApi(this.name);
```

```
201 console.log('[Platform] 当前平台: ${this.name}, api=${this.name === 'douyin' ? 'tt' : this.name ===...
202 }
203 /** 是否在小游戏环境中 */
204 get isMinigame(): boolean {
205     return this._api !== null;
206 }
207 get isWechat(): boolean {
208     return this.name === 'wechat';
209 }
210 get isDouyin(): boolean {
211     return this.name === 'douyin';
212 }
213 /** 后端 login 接口 platform 字段 (wx / dy / anon) */
214 get backendPlatformCode(): BackendPlatformCode {
215     return toBackendPlatformCode(this.name);
216 }
217 /** 是否具备 HTTP 能力 (小游戏 request 或浏览器 fetch) */
218 get canUseBackend(): boolean {
219     return typeof this._api?.request === 'function' || typeof fetch === 'function';
220 }
221 /** 开发者工具 (非真机) */
222 get isDevtools(): boolean {
223     if (!this.isMinigame) return false;
224     try {
225         return this._api?.getSystemInfoSync()?.platform === 'devtools';
226     } catch {
227         return false;
228     }
229 }
230 /** 底层 API (慎用, 优先使用封装方法) */
231 get api(): any {
232     return this._api;
233 }
234 // 存储
235 getStorageSync(key: string): string | null {
236     try {
237         return this._api?.getStorageSync(key) || null;
238     } catch (_) {
239         return null;
240     }
241 }
242 setStorageSync(key: string, value: string): void {
243     try {
244         this._api?.setStorageSync(key, value);
245     } catch (_) {}
246 }
247 /** 异步写入本地存储 (避免阻塞主线程) */
248 setStorageAsync(key: string, value: string): void {
249     try {
250         if (this._api?.setStorage) {
```

```
251     this._api.setStorage({ key, data: value, fail() {} });
252   } else {
253     this._api?.setStorageSync(key, value);
254   }
255   } catch (_) {}
256 }
257 removeStorageSync(key: string): void {
258   try {
259     this._api?.removeStorageSync(key);
260   } catch (_) {}
261 }
262 getSystemInfoSync(): Record<string, unknown> {
263   try {
264     return this._api?.getSystemInfoSync?().?? {};
265   } catch {
266     return {};
267   }
268 }
269 /** 经分 SDK / 后端 HTTP 请求 (Promise 风格) */
270 request(opts: {
271   url: string;
272   method?: 'GET' | 'POST' | 'PUT' | 'DELETE';
273   data?: unknown;
274   headers?: Record<string, string>;
275   timeoutMs?: number;
276 }): Promise<{ statusCode: number; data: unknown }> {
277   const method = (opts.method || 'POST').toUpperCase();
278   const headers: Record<string, string> = {
279     'content-type': 'application/json',
280     ...(opts.headers || {}),
281   };
282   const timeoutMs = opts.timeoutMs && opts.timeoutMs > 0 ? opts.timeoutMs : 10000;
283   const payload = opts.data === undefined || typeof opts.data === 'string'
284     ? opts.data
285     : JSON.stringify(opts.data);
286   if (this._api?.request) {
287     return new Promise((resolve, reject) => {
288       let done = false;
289       const timer = setTimeout(() => {
290         if (done) return;
291         done = true;
292         reject(new Error(`request timeout: ${opts.url}`));
293       }, timeoutMs);
294       try {
295         this._api.request({
296           url: opts.url,
297           method,
298           data: payload,
299           header: headers,
300           timeout: timeoutMs,
```

```
301     success: (res: { statusCode?: number; data?: unknown }) => {
302         if (done) return;
303         done = true;
304         clearTimeout(timer);
305         resolve({ statusCode: res?.statusCode ?? 0, data: res?.data });
306     },
307     fail: (err: unknown) => {
308         if (done) return;
309         done = true;
310         clearTimeout(timer);
311         reject(err);
312     },
313 });
314 } catch (e) {
315     if (!done) {
316         done = true;
317         clearTimeout(timer);
318         reject(e);
319     }
320 }
321 });
322 }
323 if (typeof fetch === 'function') {
324     const controller = new AbortController();
325     const timer = setTimeout(() => controller.abort(), timeoutMs);
326     return fetch(opts.url, {
327         method,
328         headers,
329         body: payload as BodyInit | undefined,
330         signal: controller.signal,
331     })
332     .then(async (res) => {
333         clearTimeout(timer);
334         const text = await res.text();
335         let data: unknown = text;
336         try { data = text ? JSON.parse(text) : null; } catch { /* keep text */ }
337         return { statusCode: res.status, data };
338     })
339     .catch((e) => {
340         clearTimeout(timer);
341         throw e;
342     });
343 }
344 return Promise.reject(new Error('no http transport available'));
345 }
346 /** 平台登录 code (wx.login / tt.login) */
347 loginCode(): Promise<string> {
348     return new Promise((resolve) => {
349         if (!this._api?.login) {
350             resolve('');
```

```
351     return;
352 }
353 try {
354     this._api.login({
355         success: (res: { code?: string }) => resolve(res?.code || ''),
356         fail: () => resolve(''),
357     });
358 } catch {
359     resolve('');
360 }
361 });
362 }
363 // 创建资源
364 /** 创建平台 Image 对象（加载本地/网络图片用） */
365 createImage(): any {
366     if (this._api?.createImage) return this._api.createImage();
367     if (typeof Image !== 'undefined') return new Image();
368     return null;
369 }
370 /** 下载远程文件到临时路径（CDN 资源用） */
371 downloadFile(url: string): Promise<{ tempFilePath?: string; statusCode?: number }> {
372     return new Promise((resolve, reject) => {
373         if (!this._api?.downloadFile) {
374             reject(new Error('downloadFile unavailable'));
375             return;
376         }
377         this._api.downloadFile({
378             url,
379             success: (res: { tempFilePath?: string; statusCode?: number }) => {
380                 const statusCode = Number(res?.statusCode || 0);
381                 if (statusCode > 0 && (statusCode < 200 || statusCode >= 300)) {
382                     reject(new Error(`downloadFile status=${statusCode} url=${url}`));
383                     return;
384                 }
385                 if (!res?.tempFilePath) {
386                     reject(new Error(`downloadFile missing tempFilePath url=${url}`));
387                     return;
388                 }
389                 resolve(res);
390             },
391             fail: (err: any) => {
392                 reject(new Error(err?.errMsg || err?.message || String(err)));
393             },
394         });
395     });
396 }
397 /** 创建 InnerAudioContext（BGM / 音效） */
398 createInnerAudioContext(): WechatMinigame.InnerAudioContext | null {
399     try {
400         return this._api?.createInnerAudioContext?.() ?? null;
```

```
401     } catch {
402         return null;
403     }
404 }
405 // 交互反馈
406 /** 短振动（消除/点击反馈），type 控制强度 */
407 vibrateShort(type: 'light' | 'medium' | 'heavy' = 'light'): void {
408     try {
409         this._api?.vibrateShort?({ type });
410     } catch (_) {}
411 }
412 /** 长振动（重击/Boss 登场等强反馈） */
413 vibrateLong(): void {
414     try {
415         this._api?.vibrateLong?();
416     } catch (_) {}
417 }
418 showToast(title: string, icon: 'success' | 'error' | 'none' = 'none'): void {
419     try {
420         this._api?.showToast?({ title, icon });
421     } catch (_) {}
422 }
423 /** 自定义分析。有宿主走 reportEvent，没有就打日志，切片验收够用。 */
424 reportEvent(name: string, data: Record<string, unknown> = {}): void {
425     try {
426         if (typeof this._api?.reportEvent === 'function') {
427             this._api.reportEvent(name, data);
428             return;
429         }
430     } catch {
431         /* 上报失败不挡玩 */
432     }
433     try {
434         console.log(`[track] ${name}`, data);
435     } catch {
436         /* */
437     }
438 }
439 /** 关掉当前原生 toast（激励广告关闭后宿主偶发自带提示时用） */
440 hideToast(): void {
441     try {
442         this._api?.hideToast?();
443     } catch (_) {}
444 }
445 /**
446  * 激励视频广告。已配置 createRewardedVideoAd 则拉起；
447  * 否则开发/本地环境短暂提示后视为成功，便于联调各广告位。
448  *
449  * 广告实例按 adUnitId 缓存并只注册一次回调：宿主的 createRewardedVideoAd 对同一
450  * adUnitId 返回同一单例，每次播放都重新 onClose 会让回调越积越多，
```

```
451  * 同一次关闭被回调 N 次（第二次以后打到已结束的 Promise 上，静默丢失奖励）。
452  */
453  showRewardedVideo(adUnitId = "): Promise<boolean> {
454    return new Promise((resolve) => {
455      // 同一时刻只允许一支广告在播，避免两个入口的奖励串到一起
456      if (this._adResolve) {
457        resolve(false);
458        return;
459      }
460      // 开发者工具里也能 new 出广告实例，但关广告经常不带 isEnded，
461      // 联调会误判没看完。工具里走桩，真机才拉真实激励视频。
462      if (this.isDevtools) {
463        this.showToast('广告播放中...');
464        setTimeout(() => {
465          this.hideToast();
466          resolve(true);
467        }, 700);
468        return;
469      }
470      let ad: any = null;
471      try {
472        ad = this._rewardedAd(adUnitId);
473      } catch (_) { /* fall through mock */ }
474      if (!ad) {
475        this.showToast('广告播放中...');
476        setTimeout(() => {
477          // 桩结束立刻清掉，避免和业务侧翻倍数额动画叠在一起
478          this.hideToast();
479          resolve(true);
480        }, 700);
481        return;
482      }
483      this._adResolve = resolve;
484      try {
485        const p = ad.show();
486        // 拉取失败先 load 再播一次：小游戏侧常见于弱网首次拉取超时
487        if (p?.catch) {
488          p.catch(() => {
489            const l = ad.load?();
490            if (l?.then) l.then(() => ad.show()).catch(() => this._settleAd(false));
491            else this._settleAd(false);
492          });
493        }
494      } catch (_) {
495        this._settleAd(false);
496      }
497    });
498  }
499  private _rewardedAd(adUnitId: string): any {
500    const cached = this._adCache.get(adUnitId);
```

```
501   if (cached) return cached;
502   const create = this._api?.createRewardedVideoAd;
503   if (typeof create !== 'function') return null;
504   const ad = create.call(this._api, { adUnitId });
505   if (!ad?.onClose || !ad?.show) return null;
506   ad.onClose((res: { isEnded?: boolean }) => this._settleAd(!res?.isEnded));
507   ad.onError?.(() => this._settleAd(false));
508   this._adCache.set(adUnitId, ad);
509   return ad;
510 }
511 private _settleAd(ok: boolean): void {
512   const resolve = this._adResolve;
513   this._adResolve = null;
514   resolve?.(ok);
515 }
516 /**
517  * 插屏广告（抖音流量主 / 广告金政策建议接入）。
518  * 未配置 adUnitId、宿主不支持、加载失败 → resolve(false)，不挡流程。
519  * 抖音要求「展示成功后再播须 destroy 重建」：每次调用新建实例，关闭后销毁。
520  */
521 showInterstitialAd(adUnitId = ""): Promise<boolean> {
522   return new Promise((resolve) => {
523     if (!adUnitId) {
524       resolve(false);
525       return;
526     }
527     const create = this._api?.createInterstitialAd;
528     if (typeof create !== 'function') {
529       resolve(false);
530       return;
531     }
532     let ad: any = null;
533     let settled = false;
534     let shown = false;
535     const finish = (ok: boolean): void => {
536       if (settled) return;
537       settled = true;
538       try { ad?.destroy?.(); } catch { /* ignore */ }
539       resolve(ok);
540     };
541     try {
542       ad = create.call(this._api, { adUnitId });
543     } catch {
544       resolve(false);
545       return;
546     }
547     if (!ad?.show) {
548       resolve(false);
549       return;
550     }
```

```
551   ad.onError?.(() => finish(false));
552   ad.onClose?.(() => finish(true));
553   ad.onLoad?.(() => {
554     try {
555       const p = ad.show();
556       if (p?.then) {
557         p.then(() => { shown = true; }).catch(() => finish(false));
558       } else {
559         shown = true;
560       }
561     } catch {
562       finish(false);
563     }
564   });
565   // 创建后会自动 load; 超时仍未展示则放行 (避免卡死跳转)
566   setTimeout(() => {
567     if (!settled && !shown) finish(false);
568   }, 5000);
569 });
570 }
571 /**
572  * 订阅消息 (抖音广告金政策建议接入) 。
573  * 必须在用户点击/支付回调里调用; tmplIds 来自后台「运营-订阅消息」。
574  * 返回各模板 id → accept|reject|ban|fail; 失败或未配置返回空对象。
575  */
576 requestSubscribeMessage(tmplIds: readonly string[]): Promise<Record<string, string>> {
577   return new Promise((resolve) => {
578     const ids = tmplIds.filter(Boolean).slice(0, 3);
579     const api = this._api?.requestSubscribeMessage;
580     if (typeof api !== 'function' || ids.length === 0) {
581       resolve({});
582       return;
583     }
584     try {
585       api.call(this._api, {
586         tmplIds: ids,
587         success: (res: Record<string, string>) => resolve(res ?? {}),
588         fail: () => resolve({}),
589         complete: () => { /* success/fail 已 settle */ },
590       });
591     } catch {
592       resolve({});
593     }
594   });
595 }
596 showModal(title: string, content: string): void {
597   try {
598     this._api?.showModal?.({ title, content, showCancel: false });
599   } catch (_) {}
600 }
```

```
601 // 分享
602 showShareMenu(opts?: { withShareTicket?: boolean; menus?: string[] }): void {
603   try {
604     this._api?.showShareMenu?({
605       withShareTicket: opts?.withShareTicket ?? true,
606       menus: opts?.menus ?? ['shareAppMessage', 'shareTimeline'],
607     });
608   } catch (_) {}
609 }
610 shareAppMessage(opts: { title: string; imageUrl?: string; query?: string }): void {
611   try {
612     this._api?.shareAppMessage?.(opts);
613   } catch (_) {}
614 }
615 onShareAppMessage(callback: () => { title: string; imageUrl?: string; query?: string }): void {
616   try {
617     this._api?.onShareAppMessage?.(callback);
618   } catch (_) {}
619 }
620 onShareTimeline(callback: () => { title: string; imageUrl?: string; query?: string }): void {
621   try {
622     this._api?.onShareTimeline?.(callback);
623   } catch (_) {}
624 }
625 // 生命周期
626 onShow(handler: (opts: any) => void): void {
627   try {
628     this._api?.onShow?.(handler);
629   } catch (_) {}
630 }
631 onHide(handler: () => void): void {
632   try {
633     this._api?.onHide?.(handler);
634   } catch (_) {}
635 }
636 // 抖音侧边栏复访
637 /** 检测宿主是否支持指定场景（如 sidebar） */
638 checkScene(opts: {
639   scene: string;
640   success?: (res: { isExist?: boolean }) => void;
641   fail?: (err?: unknown) => void;
642 }): void {
643   try {
644     if (this._api?.checkScene) {
645       this._api.checkScene(opts);
646     } else {
647       opts.fail?.({ errMsg: 'checkScene not supported' });
648     }
649   } catch (e) {
650     opts.fail?.(e);
651   }
652 }
```

```
651     }
652 }
653 /** 跳转宿主场景（侧边栏复访必接） */
654 navigateToScene(opts: {
655     scene: string;
656     success?: () => void;
657     fail?: (err?: unknown) => void;
658 }): void {
659     try {
660         if (this._api?.navigateToScene) {
661             this._api.navigateToScene(opts);
662         } else {
663             opts.fail?({ errMsg: 'navigateToScene not supported' });
664         }
665     } catch (e) {
666         opts.fail?.(e);
667     }
668 }
669 // 抖音添加到桌面（广告金政策必接）
670 /** 检查桌面快捷方式是否已添加（仅 Android 有效） */
671 checkShortcut(opts: {
672     success?: (res: { status?: { exist?: boolean; needUpdate?: boolean } }) => void;
673     fail?: (err?: unknown) => void;
674 }): void {
675     try {
676         if (this._api?.checkShortcut) {
677             this._api.checkShortcut(opts);
678         } else {
679             opts.fail?({ errMsg: 'checkShortcut not supported' });
680         }
681     } catch (e) {
682         opts.fail?.(e);
683     }
684 }
685 /**
686  * 添加小游戏到手机桌面（须在用户点击/touchend 内同步调用）
687  * 仅支持抖音 / 抖音极速版 / 抖音火山版
688  */
689 addShortcut(opts: {
690     success?: () => void;
691     fail?: (err?: { errMsg?: string }) => void;
692     complete?: () => void;
693 }): void {
694     try {
695         if (this._api?.addShortcut) {
696             this._api.addShortcut(opts);
697         } else {
698             opts.fail?({ errMsg: 'addShortcut not supported' });
699         }
700     } catch (e) {
```

```

701     opts.fail?.(e as { errMsg?: string });
702   }
703 }
704 /**
705  * 加载本地自定义字体。
706  * 优先 wx/tt.loadFont(path) (同步返回 family) ; 否则走 loadFontFace。
707  * 非小游戏环境返回 null, 由 FontService 用 @font-face 兜底。
708  */
709 loadFont(path: string, family: string): Promise<string | null> {
710   if (!this.isMinigame) return Promise.resolve(null);
711   // 微信/抖音小游戏: loadFont 最稳, 直接返回可用 family 名
712   if (typeof this._api?.loadFont === 'function') {
713     try {
714       const loaded = this._api.loadFont(path) as string | undefined;
715       return Promise.resolve(loaded || family);
716     } catch (e) {
717       console.warn('[Platform] loadFont 失败', path, e);
718     }
719   }
720   if (typeof this._api?.loadFontFace === 'function') {
721     return new Promise((resolve) => {
722       try {
723         this._api.loadFontFace({
724           family,
725           source: `url("${path}")`,
726           global: true,
727           success: () => resolve(family),
728           fail: (err: unknown) => {
729             console.warn('[Platform] loadFontFace 失败', path, err);
730             resolve(null);
731           },
732         });
733       } catch (e) {
734         console.warn('[Platform] loadFontFace 异常', path, e);
735         resolve(null);
736       }
737     });
738   }
739   return Promise.resolve(null);
740 }
741 }
742 export const Platform = new PlatformServiceClass();
743 // ===== FILE: src/core/BackendService.ts =====
744 import {
745   BACKEND_ANON_ID_KEY,
746   BACKEND_BASE_URL,
747   BACKEND_LOGIN_PATH,
748   BACKEND_PULL_PATH,
749   BACKEND_PUSH_PATH,
750   BACKEND_REQUEST_TIMEOUT_MS,

```

```
751  BACKEND_TOKEN_KEY,
752  } from '@config/CloudConfig';
753  import { Platform } from '@core/PlatformService';
754  export interface BackendPullResult {
755    userId: string;
756    platform: string;
757    exists: boolean;
758    schemaVersion: number;
759    updatedAt: number;
760    payload: Record<string, string>;
761    payloadKeys: string[];
762    clientFingerprint?: string;
763  }
764  export interface BackendPushPayload {
765    schemaVersion: number;
766    updatedAt: number;
767    baseRemoteUpdatedAt: number;
768    clientFingerprint: string;
769    payload: Record<string, string>;
770  }
771  export interface BackendPushResult {
772    userId: string;
773    updatedAt: number;
774    savedAt: number;
775    mode: 'insert' | 'update';
776    sizeBytes: number;
777  }
778  interface StoredToken {
779    token: string;
780    userId: string;
781    platform: string;
782    expiresAt: number;
783  }
784  export class BackendError extends Error {
785    readonly status: number;
786    readonly code: string;
787    readonly data?: unknown;
788    constructor(status: number, code: string, message: string, data?: unknown) {
789      super(message);
790      this.name = 'BackendError';
791      this.status = status;
792      this.code = code;
793      this.data = data;
794    }
795  }
796  class BackendServiceClass {
797    private stored: StoredToken | null = null;
798    private loginInflight: Promise<StoredToken> | null = null;
799    get available(): boolean {
800      return Platform.canUseBackend;
```

```
801 }
802 get userId(): string {
803     return this.stored?.userId || '';
804 }
805 async ensureToken(): Promise<StoredToken> {
806     if (this.stored && this.stored.expiresAt - Date.now() > 60_000) {
807         if (this.isTokenForCurrentPlatform(this.stored)) {
808             return this.stored;
809         }
810         console.warn('[Backend] 缓存 token 平台不匹配(${this.stored.platform}→${this.expectedPlatform()}), ...
811         this.clearToken();
812     }
813     const cached = this.loadTokenFromStorage();
814     if (cached && cached.expiresAt - Date.now() > 60_000) {
815         if (this.isTokenForCurrentPlatform(cached)) {
816             this.stored = cached;
817             return cached;
818         }
819         console.warn('[Backend] 本地 token 平台不匹配(${cached.platform}→${this.expectedPlatform()}), 重新登录');
820         this.clearToken();
821     }
822     if (this.loginInflight) {
823         return this.loginInflight;
824     }
825     this.loginInflight = this.login().finally(() => {
826         this.loginInflight = null;
827     });
828     return this.loginInflight;
829 }
830 pullSave(): Promise<BackendPullResult> {
831     return this.callWithAuth<BackendPullResult>(BACKEND_PULL_PATH, {});
832 }
833 pushSave(snapshot: BackendPushPayload): Promise<BackendPushResult> {
834     return this.callWithAuth<BackendPushResult>(BACKEND_PUSH_PATH, snapshot);
835 }
836 clearToken(): void {
837     this.stored = null;
838     Platform.removeStorageSync(BACKEND_TOKEN_KEY);
839 }
840 private async login(): Promise<StoredToken> {
841     const body = await this.buildLoginBody();
842     console.log('[Backend] login start platform=${body.platform}');
843     const { status, data } = await this.request(BACKEND_LOGIN_PATH, body, undefined);
844     const res = data as { ok?: boolean; code?: string; error?: string; data?: Partial<StoredToken...
845     if (status !== 200 || !res || res.ok !== true || !res.data?.token) {
846         const code = res?.code || 'LOGIN_FAIL';
847         const msg = res?.error || `login failed (status=${status})`;
848         console.warn('[Backend] login fail platform=${body.platform} code=${code} msg=${msg}');
849         throw new BackendError(status, code, msg, res?.data);
850     }
```

```
851   const stored = {
852     token: String(res.data.token),
853     userId: String(res.data.userId || ''),
854     platform: String(res.data.platform || body.platform),
855     expiresAt: Number(res.data.expiresAt || 0),
856   };
857   this.stored = stored;
858   Platform.setStorageSync(BACKEND_TOKEN_KEY, JSON.stringify(stored));
859   console.log(`[Backend] login ok platform=${stored.platform} userId=${stored.userId}`);
860   return stored;
861 }
862 private expectedPlatform(): string {
863   return Platform.backendPlatformCode;
864 }
865 private isTokenForCurrentPlatform(stored: StoredToken): boolean {
866   const expected = this.expectedPlatform();
867   const actual = stored.platform || stored.userId.split(':')[0] || '';
868   return actual === expected;
869 }
870 private async buildLoginBody(): Promise<{ platform: string; code?: string; anonId?: string }> {
871   const platform = Platform.backendPlatformCode;
872   if (platform === 'wx' || platform === 'dy') {
873     const code = await Platform.loginCode();
874     if (!code) {
875       const errCode = platform === 'wx' ? 'NO_WX_CODE' : 'NO_TT_CODE';
876       throw new BackendError(0, errCode, `${platform} login did not return code`);
877     }
878     return { platform, code };
879   }
880   return { platform: 'anon', anonId: this.getOrCreateAnonId() };
881 }
882 private getOrCreateAnonId(): string {
883   const existing = Platform.getStorageSync(BACKEND_ANON_ID_KEY);
884   if (existing) {
885     return existing;
886   }
887   const id = `anon_${Date.now().toString(36)}_${Math.random().toString(36).slice(2, 10)}`;
888   Platform.setStorageSync(BACKEND_ANON_ID_KEY, id);
889   return id;
890 }
891 private async callWithAuth<T>(path: string, body: unknown): Promise<T> {
892   const token = await this.ensureToken();
893   const { status, data } = await this.request(path, body, token.token);
894   if (status === 401) {
895     this.clearToken();
896     const retryToken = await this.ensureToken();
897     const retry = await this.request(path, body, retryToken.token);
898     return this.unwrap<T>(retry.status, retry.data);
899   }
900   return this.unwrap<T>(status, data);
```

```
901 }
902 private unwrap<T>(status: number, data: unknown): T {
903     const res = data as { ok?: boolean; code?: string; error?: string; data?: T };
904     if (status === 200 && res?.ok === true) {
905         return res.data as T;
906     }
907     const code = res?.code || `HTTP_${status}`;
908     const msg = res?.error || `request failed status=${status}`;
909     throw new BackendError(status, code, msg, res?.data);
910 }
911 private async request(
912     path: string,
913     body: unknown,
914     token: string | undefined,
915 ): Promise<{ status: number; data: unknown }> {
916     const headers: Record<string, string> = {
917         'content-type': 'application/json',
918     };
919     if (token) {
920         headers.authorization = `Bearer ${token}`;
921     }
922     const res = await Platform.request({
923         url: BACKEND_BASE_URL + path,
924         method: 'POST',
925         data: body || {},
926         headers,
927         timeoutMs: BACKEND_REQUEST_TIMEOUT_MS,
928     });
929     return { status: res.statusCode, data: res.data };
930 }
931 private loadTokenFromStorage(): StoredToken | null {
932     const raw = Platform.getStorageSync(BACKEND_TOKEN_KEY);
933     if (!raw) {
934         return null;
935     }
936     try {
937         const parsed = JSON.parse(raw) as Partial<StoredToken>;
938         if (!parsed.token) {
939             return null;
940         }
941         return {
942             token: parsed.token,
943             userId: parsed.userId || '',
944             platform: parsed.platform || '',
945             expiresAt: Number(parsed.expiresAt || 0),
946         };
947     } catch {
948         return null;
949     }
950 }
```

```
951 }
952 export const BackendService = new BackendServiceClass();
953 // ===== FILE: src/core/CloudSyncManager.ts =====
954 import {
955   CLOUD_SYNC_BASE_DELAY_MS,
956   CLOUD_SYNC_DEBOUNCE_MS,
957   CLOUD_SYNC_LOG_THRESHOLD,
958   CLOUD_SYNC_MAX_BACKOFF_MS,
959   CLOUD_SYNC_MAX_FAIL_COUNT,
960   CLOUD_SYNC_RETRY_INTERVAL_MS,
961   CLOUD_SYNC_STARTUP_TIMEOUT_MS,
962 } from '@config/CloudConfig';
963 import { BackendError, BackendService } from '@core/BackendService';
964 import { PersistService } from '@core/PersistService';
965 import { Platform } from '@core/PlatformService';
966 export type CloudAuthorityState = 'disabled' | 'unknown' | 'confirmedRemote' | 'cacheOnly';
967 export interface CloudStartupSyncResult {
968   status: 'disabled' | 'confirmed' | 'remote-applied' | 'cache-only';
969   reason: string;
970 }
971 class CloudSyncManagerClass {
972   private initPromise: Promise<void> | null = null;
973   private startupPromise: Promise<void> | null = null;
974   private cloudReady = false;
975   private initDone = false;
976   private syncTimer: ReturnType<typeof setTimeout> | null = null;
977   private retryTimer: ReturnType<typeof setInterval> | null = null;
978   private syncFailCount = 0;
979   private syncDisabled = false;
980   private syncing = false;
981   private syncPending = false;
982   private authorityState: CloudAuthorityState = this.enabled ? 'unknown' : 'disabled';
983   private lastStartupRemoteApplied = false;
984   constructor() {
985     PersistService.subscribe((changedKeys) => {
986       if (changedKeys.length > 0) {
987         this.scheduleSync(`dirty:${changedKeys.length}`);
988       }
989     });
990   }
991   get enabled(): boolean {
992     return BackendService.available;
993   }
994   get ready(): boolean {
995     return this.cloudReady;
996   }
997   get userId(): string {
998     return BackendService.userId;
999   }
1000   prewarm(): void {
```

```
1001   if (!this.enabled) {
1002       return;
1003   }
1004   if (!this.startupPromise) {
1005       this.startupPromise = this.initialize();
1006   }
1007   }
1008   async awaitStartupSync(timeoutMs = CLOUD_SYNC_STARTUP_TIMEOUT_MS): Promise<CloudStartupSyncResu...
1009   if (!this.enabled) {
1010       this.authorityState = 'disabled';
1011       return { status: 'disabled', reason: 'backend-disabled' };
1012   }
1013   this.prewarm();
1014   if (!this.startupPromise) {
1015       return { status: 'disabled', reason: 'startup-missing' };
1016   }
1017   let timer: ReturnType<typeof setTimeout> | null = null;
1018   const result = await Promise.race([
1019       this.startupPromise.then(() => 'done' as const).catch(() => 'done' as const),
1020       new Promise<void>((resolve) => {
1021           timer = setTimeout(resolve, timeoutMs);
1022       }).then(() => 'timeout' as const),
1023   ]);
1024   if (timer) {
1025       clearTimeout(timer);
1026   }
1027   if (result === 'timeout') {
1028       return { status: 'cache-only', reason: 'startup-timeout' };
1029   }
1030   if (this.authorityState === 'cacheOnly') {
1031       return { status: 'cache-only', reason: 'startup-pull-failed' };
1032   }
1033   return {
1034       status: this.lastStartupRemoteApplied ? 'remote-applied' : 'confirmed',
1035       reason: this.lastStartupRemoteApplied ? 'remote-imported' : 'cloud-confirmed',
1036   };
1037   }
1038   scheduleSync(reason = 'debounce'): void {
1039       if (!this.enabled) {
1040           return;
1041       }
1042       this.prewarm();
1043       if (this.authorityState === 'cacheOnly') {
1044           this.syncPending = true;
1045           return;
1046       }
1047       if (!this.initDone) {
1048           this.syncPending = true;
1049           return;
1050       }
1051   }
```

```
1051   if (!this.cloudReady || this.syncDisabled) {
1052     return;
1053   }
1054   if (this.syncTimer) {
1055     clearTimeout(this.syncTimer);
1056   }
1057   const delay = this.syncFailCount > 0
1058     ? Math.min(CLOUD_SYNC_BASE_DELAY_MS * Math.pow(2, this.syncFailCount - 1), CLOUD_SYNC_MAX_B...
1059       : CLOUD_SYNC_DEBOUNCE_MS;
1060   this.syncTimer = setTimeout(() => {
1061     this.syncTimer = null;
1062     void this.syncToCloud(reason);
1063   }, delay);
1064 }
1065 async flushNow(reason = 'manual'): Promise<void> {
1066   if (!this.enabled) {
1067     return;
1068   }
1069   this.prewarm();
1070   if (this.startupPromise) {
1071     try {
1072       await this.startupPromise;
1073     } catch {
1074       // 初始化失败已经走本地缓存
1075     }
1076   }
1077   if (this.authorityState === 'cacheOnly' || !this.cloudReady) {
1078     return;
1079   }
1080   if (this.syncTimer) {
1081     clearTimeout(this.syncTimer);
1082     this.syncTimer = null;
1083   }
1084   await this.syncToCloud(reason, true);
1085 }
1086 private async initialize(): Promise<void> {
1087   if (this.initPromise) {
1088     return this.initPromise;
1089   }
1090   this.initPromise = (async () => {
1091     try {
1092       await BackendService.ensureToken();
1093       this.cloudReady = !!BackendService.userId;
1094       if (!this.cloudReady) {
1095         this.enterCacheOnly('no-user-id');
1096         return;
1097       }
1098       await this.pullFromCloudOnStartup();
1099     } catch (error) {
1100       console.warn('[CloudSync] init failed, using local cache', error);
```

```
1101     this.enterCacheOnly('init-failed');
1102   } finally {
1103     this.initDone = true;
1104     if (this.syncPending && this.cloudReady) {
1105       this.syncPending = false;
1106       this.scheduleSync('pending-after-init');
1107     }
1108   }
1109   })();
1110   return this.initPromise;
1111 }
1112 private async pullFromCloudOnStartup(): Promise<void> {
1113   let remote;
1114   try {
1115     remote = await BackendService.pullSave();
1116   } catch (error) {
1117     console.warn('[CloudSync] startup pull failed, keeping local cache', error);
1118     this.enterCacheOnly('startup-pull-failed');
1119     return;
1120   }
1121   const hasLocalData = PersistService.hasAnyLocalCloudData();
1122   if (!remote.exists || Object.keys(remote.payload || {}).length === 0) {
1123     // 远端空档: 保留本地进度, 标脏后推上去。不能按空 payload 覆盖本地。
1124     this.confirmRemoteBaseline(0, remote.exists ? 'startup-empty-remote-doc' : 'startup-no-remo...
1125     this.lastStartupRemoteApplied = false;
1126     if (hasLocalData) {
1127       PersistService.touchCloudMeta();
1128       this.scheduleSync('startup-seed-remote');
1129     }
1130     return;
1131   }
1132   const localMeta = PersistService.getCloudSyncMeta();
1133   const remoteUpdatedAt = Number(remote.updatedAt) || 0;
1134   const hasKnownRemoteBaseline = localMeta.remoteUpdatedAt > 0;
1135   const shouldApplyRemote = !hasLocalData || !hasKnownRemoteBaseline || remoteUpdatedAt > local...
1136   if (shouldApplyRemote) {
1137     PersistService.importCloudSnapshot({
1138       updatedAt: remoteUpdatedAt,
1139       payload: remote.payload || {},
1140       reason: 'startup',
1141     });
1142     this.confirmRemoteBaseline(remoteUpdatedAt, 'startup-remote-imported');
1143     this.lastStartupRemoteApplied = true;
1144     return;
1145   }
1146   this.confirmRemoteBaseline(remoteUpdatedAt, 'startup-remote-confirmed');
1147   if (PersistService.isCloudDirty()) {
1148     this.scheduleSync('startup-local-dirty');
1149   }
1150 }
```

```
1151 private async syncToCloud(reason: string, force = false): Promise<void> {
1152   if (!this.cloudReady || (!force && this.syncDisabled) || this.authorityState === 'cacheOnly') {
1153     return;
1154   }
1155   if (this.syncing) {
1156     this.syncPending = true;
1157     return;
1158   }
1159   this.syncing = true;
1160   try {
1161     const snapshot = PersistService.exportCloudSnapshot();
1162     if (!PersistService.isCloudDirty() || snapshot.payloadKeys.length === 0) {
1163       return;
1164     }
1165     if (snapshot.updatedAt <= 0) {
1166       PersistService.touchCloudMeta();
1167     }
1168     const finalSnapshot = PersistService.exportCloudSnapshot();
1169     try {
1170       const res = await BackendService.pushSave({
1171         schemaVersion: finalSnapshot.schemaVersion,
1172         updatedAt: finalSnapshot.updatedAt,
1173         baseRemoteUpdatedAt: finalSnapshot.baseRemoteUpdatedAt,
1174         clientFingerprint: this.buildClientFingerprint(),
1175         payload: finalSnapshot.payload,
1176       });
1177       PersistService.markCloudSynced(res.updatedAt || finalSnapshot.updatedAt);
1178       this.confirmRemoteBaseline(res.updatedAt || finalSnapshot.updatedAt, `push-ok:${reason}`);
1179       this.syncFailCount = 0;
1180       this.syncDisabled = false;
1181       if (this.retryTimer) {
1182         clearInterval(this.retryTimer);
1183         this.retryTimer = null;
1184       }
1185     } catch (error) {
1186       if (error instanceof BackendError && error.code === 'STALE_UPDATE' && error.data && typeo...
1187         const remote = (error.data as { remote?: { updatedAt?: number; payload?: Record<string,...
1188         PersistService.importCloudSnapshot({
1189           updatedAt: Number(remote?.updatedAt) || Date.now(),
1190           payload: remote?.payload || {},
1191           reason: 'stale-update',
1192         });
1193         this.confirmRemoteBaseline(Number(remote?.updatedAt) || Date.now(), 'stale-update');
1194         this.syncFailCount = 0;
1195         this.syncDisabled = false;
1196         return;
1197       }
1198       throw error;
1199     }
1200   } catch (error: unknown) {
```

```

1201     this.syncFailCount += 1;
1202     const message = error instanceof Error ? error.message : String(error);
1203     if (this.syncFailCount <= CLOUD_SYNC_LOG_THRESHOLD) {
1204         console.warn('[CloudSync] push failed (${this.syncFailCount}/${CLOUD_SYNC_MAX_FAIL_COUNT})...
1205     }
1206     if (this.syncFailCount >= CLOUD_SYNC_MAX_FAIL_COUNT) {
1207         this.syncDisabled = true;
1208         if (!this.retryTimer) {
1209             this.retryTimer = setInterval(() => {
1210                 if (!this.syncing && PersistService.isCloudDirty()) {
1211                     void this.syncToCloud('retry-interval', true);
1212                 }
1213             }, CLOUD_SYNC_RETRY_INTERVAL_MS);
1214         }
1215     } else if (PersistService.isCloudDirty()) {
1216         this.scheduleSync('retry-after-fail:${reason}');
1217     }
1218     } finally {
1219         this.syncing = false;
1220         if (this.syncPending && !this.syncDisabled) {
1221             this.syncPending = false;
1222             this.scheduleSync('pending-resume');
1223         }
1224     }
1225 }
1226 private buildClientFingerprint(): string {
1227     const info = Platform.getSystemInfoSync();
1228     return [
1229         Platform.name,
1230         info.brand,
1231         info.model,
1232         info.platform,
1233     ].map((v) => (v == null ? '' : String(v))).filter(Boolean).join('|').slice(0, 160);
1234 }
1235 private enterCacheOnly(reason: string): void {
1236     if (this.authorityState === 'cacheOnly') {
1237         return;
1238     }
1239     this.authorityState = 'cacheOnly';
1240     console.warn('[CloudSync] cacheOnly reason=${reason}');
1241 }
1242 private confirmRemoteBaseline(remoteUpdatedAt: number, reason: string): void {
1243     this.authorityState = 'confirmedRemote';
1244     console.log('[CloudSync] remote baseline confirmed reason=${reason}, remoteUpdatedAt=${remote...
1245 }
1246 }
1247 export const CloudSyncManager = new CloudSyncManagerClass();
1248 // ===== FILE: src/core/RunMemory.ts =====
1249 /**
1250  * 存档: 村民名单、进化、星级、村庄等级、四种资源、弹子、推图进度、布阵。

```

```
1251 *
1252 * 一条硬口径（§5）：**货币只有四种** —— 废铁 / 零件 / 工分 / 村庄经验。
1253 * 弹子是次数不是货币，所以它跟资源分开存，也不进货币条。
1254 * 想加第五种之前先回去看反目标第五条。
1255 *
1256 * 这一版把上一版的字段全换了（改装件、门路研发、难度阶梯、废品堆全部下线），
1257 * 所以老存档不迁移、直接重置：`REV` 不匹配就当新档。
1258 * 迁移的成本远高于收益 —— 上一版的养成对象（27 件破烂的星级）
1259 * 在这一版压根不存在，硬折算只会折出一个玩家看不懂的开局。
1260 */
1261 import { SAVE_KEY } from '@config/CloudConfig';
1262 import { PersistService } from '@core/PersistService';
1263 import {
1264   PELLET_AD, PELLET_AD_DAILY, PELLET_CLEAR, PELLET_FIRST, PELLET_LOSE,
1265   SETTLE_SCRAP, SETTLE_SCRAP_REPLAY, creditPity, mulberry32, pelletCap,
1266   pelletRegenMin, shoot,
1267   type ShotResult,
1268 } from '@balance/stall';
1269 import {
1270   CALL_COST, addVillageExp, clampVillageLv, craftOf, evoFromCraft,
1271   nextFeed, rollCall, squadCap, starsOf, yieldMul,
1272   type Progress,
1273 } from '@balance/village';
1274 import {
1275   DEFAULT_SQUAD, STAR_MAX, VILLAGERS, VILLAGER_BY_ID,
1276 } from '@balance/villagers';
1277 import { STAGE_COUNT, clampStage, getStage } from '@balance/stages';
1278 import { CELL_COUNT, LANE_COUNT } from '@balance/combat';
1279 const KEY = SAVE_KEY;
1280 /** 存档格式版本。改字段就往上加一，老档直接重置 */
1281 const REV = 3;
1282 /** 战场上一个人的位置。存下来是为了下一关不用重排 */
1283 export interface Slot {
1284   id: string;
1285   lane: number;
1286   cell: number;
1287 }
1288 export interface RunMemory {
1289   rev: number;
1290   /* ---- 养成 ---- */
1291   villageLv: number;
1292   villageExp: number;
1293   /** 已入伙的村民 id，按入伙顺序 */
1294   roster: string[];
1295   /** 每人几阶（1~3）。由手艺推导，存着给旧档和面板用 */
1296   evo: Record<string, number>;
1297   /** 每人手艺 1~10。缺字段时按 evo 反推 */
1298   craft: Record<string, number>;
1299   /** 每人几颗星 */
1300   stars: Record<string, number>;
```

```
1301 scrap: number;
1302 parts: number;
1303 credits: number;
1304 /* ---- 弹弓摊 ---- */
1305 /** 手上还有几发 */
1306 pellets: number;
1307 /** 上次结算离线回弹的时刻 */
1308 pelletAtMs: number;
1309 /** 摊子的工分保底计数（离上次出工分过了几发） */
1310 stallPity: number;
1311 /** 今天摊子广告看了几次，以及那是哪一天 */
1312 adCount: number;
1313 adDay: string;
1314 /* ---- 推图 ---- */
1315 stageId: number;
1316 stageTop: number;
1317 /** 每关拿到过的最高星评 */
1318 stageStars: Record<number, number>;
1319 /* ---- 布阵 ---- */
1320 /** 上一次的排法。开下一关时直接铺上 */
1321 layout: Slot[];
1322 /* ---- 其他 ---- */
1323 /** 一共喊过几次人。前 4 次必出新人的保底靠它 */
1324 callCount: number;
1325 /** 见过的人，点亮图鉴用（含已经不在名单里的，虽然目前不会掉人） */
1326 seenIds: string[];
1327 }
1328 function empty(): RunMemory {
1329     return {
1330         rev: REV,
1331         villageLv: 1,
1332         villageExp: 0,
1333         roster: [...DEFAULT_SQUAD],
1334         evo: {},
1335         craft: {},
1336         stars: {},
1337         scrap: 0,
1338         parts: 0,
1339         credits: 0,
1340         pellets: 6,
1341         pelletAtMs: Date.now(),
1342         stallPity: 0,
1343         adCount: 0,
1344         adDay: "",
1345         stageId: 1,
1346         stageTop: 1,
1347         stageStars: {},
1348         layout: [],
1349         callCount: 0,
1350         seenIds: [...DEFAULT_SQUAD],
```

```
1351   };
1352 }
1353 function validIds(raw: unknown, fallback: readonly string[]): string[] {
1354   if (!Array.isArray(raw)) return [...fallback];
1355   const out = raw.filter(
1356     (id): id is string => typeof id === 'string' && VILLAGER_BY_ID[id] !== undefined,
1357   );
1358   return out.length > 0 ? [...new Set(out)] : [...fallback];
1359 }
1360 function numMap(raw: unknown, lo: number, hi: number): Record<string, number> {
1361   const out: Record<string, number> = {};
1362   if (!raw || typeof raw !== 'object') return out;
1363   for (const [k, v] of Object.entries(raw as Record<string, unknown>)) {
1364     if (!VILLAGER_BY_ID[k]) continue;
1365     const n = Math.floor(Number(v) || 0);
1366     if (n > lo) out[k] = Math.min(hi, n);
1367   }
1368   return out;
1369 }
1370 function validLayout(raw: unknown, roster: readonly string[]): Slot[] {
1371   if (!Array.isArray(raw)) return [];
1372   const owned = new Set<string>(roster);
1373   const used = new Set<string>();
1374   const out: Slot[] = [];
1375   for (const r of raw) {
1376     const s = r as Partial<Slot>;
1377     if (typeof s.id !== 'string' || !owned.has(s.id)) continue;
1378     const lane = Math.floor(Number(s.lane) || 0);
1379     const cell = Math.floor(Number(s.cell) || 0);
1380     if (lane < 0 || lane >= LANE_COUNT || cell < 0 || cell >= CELL_COUNT) continue;
1381     const key = `${lane},${cell}`;
1382     if (used.has(key) || out.some(o => o.id === s.id)) continue;
1383     used.add(key);
1384     out.push({ id: s.id, lane, cell });
1385   }
1386   return out;
1387 }
1388 export function loadMemory(): RunMemory {
1389   try {
1390     const raw = PersistService.readRaw(KEY);
1391     if (!raw) return empty();
1392     const p = JSON.parse(raw) as Partial<RunMemory>;
1393     // 上一版的存档字段这一版全没有，硬折算只会折出看不懂的开局
1394     if (Number(p.rev) !== REV) return empty();
1395     const roster = validIds(p.roster, DEFAULT_SQUAD);
1396     const stageTop = clampStage(p.stageTop);
1397     const mem: RunMemory = {
1398       rev: REV,
1399       villageLv: clampVillageLv(p.villageLv),
1400       villageExp: Math.max(0, Number(p.villageExp) || 0),
```

```

1401     roster,
1402     evo: numMap(p.evo, 1, 3),
1403     craft: numMap(p.craft, 1, 10),
1404     stars: numMap(p.stars, 0, STAR_MAX),
1405     scrap: Math.max(0, Number(p.scrap) || 0),
1406     parts: Math.max(0, Number(p.parts) || 0),
1407     credits: Math.max(0, Number(p.credits) || 0),
1408     pellets: Math.max(0, Number(p.pellets) || 0),
1409     pelletAtMs: Math.max(0, Number(p.pelletAtMs) || Date.now()),
1410     stallPity: Math.max(0, Number(p.stallPity) || 0),
1411     adCount: Math.max(0, Number(p.adCount) || 0),
1412     adDay: typeof p.adDay === 'string' ? p.adDay : '',
1413     stageTop,
1414     stageId: Math.min(clampStage(p.stageId), stageTop),
1415     stageStars: (() => {
1416         const out: Record<number, number> = {};
1417         const src = p.stageStars;
1418         if (src && typeof src === 'object') {
1419             for (const [k, v] of Object.entries(src as Record<string, unknown>)) {
1420                 const id = Number(k);
1421                 if (id >= 1 && id <= STAGE_COUNT) {
1422                     out[id] = Math.max(0, Math.min(3, Math.floor(Number(v) || 0)));
1423                 }
1424             }
1425         }
1426         return out;
1427     })(),
1428     layout: validLayout(p.layout, roster),
1429     callCount: Math.max(0, Number(p.callCount) || 0),
1430     seenIds: validIds(p.seenIds, roster),
1431 };
1432 return mem;
1433 } catch {
1434     return empty();
1435 }
1436 }
1437 function persist(next: RunMemory): RunMemory {
1438     const out = { ...next, rev: REV };
1439     try {
1440         PersistService.writeRaw(KEY, JSON.stringify(out));
1441     } catch {
1442         /* 模拟器偶发写失败，不挡继续玩 */
1443     }
1444     return out;
1445 }
1446 /** 存档摊成引擎和面板都能读的 Progress */
1447 export function progressOf(mem: RunMemory): Progress {
1448     return {
1449         villageLv: mem.villageLv,
1450         villageExp: mem.villageExp,

```

```
1451   roster: mem.roster,
1452   evo: mem.evo,
1453   craft: mem.craft,
1454   stars: mem.stars,
1455   scrap: mem.scrap,
1456   parts: mem.parts,
1457   credits: mem.credits,
1458 };
1459 }
1460 /** 这一局能上几个人 */
1461 export function capOf(mem: RunMemory): number {
1462   return squadCap(mem.villageLv);
1463 }
1464 /* ----- 弹子 ----- */
1465 /**
1466  * 把离线攒的弹子结算到现在。
1467  *
1468  * 只回到 pelletCap, 不许越过 —— 弹子是次数, 攒着不玩没有额外好处,
1469  * 这是「每天回来打一会儿」和「攒一周一次性刷完」之间的那道闸。
1470  */
1471 export function settlePellets(nowMs: number = Date.now()): RunMemory {
1472   const prev = loadMemory();
1473   const cap = pelletCap(prev.villageLv);
1474   if (prev.pelletAtMs <= 0) return persist({ ...prev, pelletAtMs: nowMs });
1475   const perMs = pelletRegenMin(prev.villageLv) * 60_000;
1476   const gained = Math.floor((nowMs - prev.pelletAtMs) / perMs);
1477   if (gained <= 0) return prev;
1478   const pellets = Math.min(cap, prev.pellets + gained);
1479   return persist({
1480     ...prev,
1481     pellets,
1482     // 余数留着, 别让频繁进出把零头抹掉
1483     pelletAtMs: prev.pelletAtMs + gained * perMs,
1484   });
1485 }
1486 /** 打一发。没弹子返回 undefined */
1487 export function shootStall(nowMs: number = Date.now()): {
1488   mem: RunMemory;
1489   result: ShotResult;
1490 } | undefined {
1491   const prev = settlePellets(nowMs);
1492   if (prev.pellets <= 0) return undefined;
1493   const rng = mulberry32((nowMs ^ (prev.pellets * 2654435761)) >>> 0);
1494   const { result, pityCount } = shoot(rng, prev.villageLv, prev.stallPity);
1495   const grown = addVillageExp(progressOf(prev), Math.round(result.gain.exp));
1496   const mem = persist({
1497     ...prev,
1498     pellets: prev.pellets - 1,
1499     stallPity: pityCount,
1500     scrap: prev.scrap + Math.round(result.gain.scrap),
```

```
21384 let titleCy = (plateTop + slotTop) / 2 + 12;
21385 if (titleCy + glyphH / 2 > slotTop - 10) {
21386   titleCy = slotTop - 10 - glyphH / 2;
21387 }
21388 if (titleCy - glyphH / 2 < 8) {
21389   glyphH = Math.max(32, (titleCy - 8) * 2);
21390 }
21391 return {
21392   titleH,
21393   title: { cx: 375, cy: titleCy, w: 750, h: titleH },
21394   titleGlyphH: glyphH,
21395   exp: { x: 78, y: slotMid, w: 594 },
21396   hintY: slotTop + titleH * 0.12,
21397   stamp: {
21398     y: titleH * 0.71,
21399     w: 750 * 0.20,
21400     h: titleH * 0.20,
21401     cxs: [750 * 0.141, 750 * 0.379, 750 * 0.615, 750 * 0.854],
21402   },
21403   barBottom: titleH,
21404 };
21405 }
21406 /**
21407  * 编队 / 战斗顶板。底只铺切下来的上半块锈铁，
21408  * 上场 / 来 / 波 等字和锈章是另出的图，叠在板上，不烤进底板。
21409  *
21410  * 切图是 1280×398，750 宽时约 233 高；为了塞下三枚章会略往下拉。
21411  */
21412 export interface BattleHudLay {
21413   titleH: number;
21414   title: { cx: number; cy: number };
21415   titleGlyphH: number;
21416   hintY: number;
21417   stamp: { y: number; w: number; h: number; cxs: readonly number[] };
21418   barBottom: number;
21419 }
21420 export function battleHudLay(safeTop: number, height: number): BattleHudLay {
21421   const safe = Math.max(safeTop, 16);
21422   let titleH = Math.round(750 * 398 / 1280);
21423   if (titleH < safe + 200) titleH = Math.min(Math.round(height * 0.24), safe + 220);
21424   if (titleH > height * 0.24) titleH = Math.round(height * 0.24);
21425   const titleGlyphH = Math.max(32, Math.round(titleH * 0.16));
21426   let titleCy = titleH * 0.24;
21427   if (titleCy - titleGlyphH / 2 < safe + 4) {
21428     titleCy = safe + 4 + titleGlyphH / 2;
21429   }
21430   const hintY = titleH - 16;
21431   const stampH = Math.round(titleH * 0.30);
21432   const stampY = hintY - 12 - stampH / 2;
21433   return {
```

```

21434   titleH,
21435   title: { cx: 375, cy: titleCy },
21436   titleGlyphH,
21437   hintY,
21438   stamp: {
21439     y: stampY,
21440     w: 200,
21441     h: stampH,
21442     cxs: [155, 375, 595],
21443   },
21444   barBottom: titleH,
21445 };
21446 }
21447 /**
21448  * 弹弓摊顶板。底图跟战斗同一张切下来的锈铁，字另叠。
21449  * 弹子对准底牌本身的上下中线（板顶到口袋上沿），别按胶囊下沿把字往下推。
21450  * 口袋钉在锈铁下沿。750 宽时按原图比例大约 233 高。
21451  */
21452 export interface StallHudLay {
21453   titleH: number;
21454   title: { cx: number; cy: number };
21455   titleGlyphH: number;
21456   hintY: number;
21457   pocket: { y: number; w: number; h: number; cxs: readonly [number, number, number] };
21458   barBottom: number;
21459 }
21460 export function stallHudLay(safeTop: number, height: number): StallHudLay {
21461   const safe = Math.max(safeTop, 16);
21462   let titleH = Math.round(750 * 398 / 1280);
21463   if (titleH < safe + 96) titleH = Math.min(Math.round(height * 0.18), safe + 140);
21464   if (titleH > height * 0.2) titleH = Math.round(height * 0.2);
21465   const padB = 12;
21466   const pocketH = 44;
21467   const pocketY = titleH - padB - pocketH / 2;
21468   const pocketTop = pocketY - pocketH / 2;
21469   const titleGlyphH = Math.max(36, Math.round(titleH * 0.22));
21470   const hintH = 22;
21471   const gap1 = 6;
21472   const plateTop = titleH * 0.10;
21473   let titleCy = (plateTop + pocketTop) / 2;
21474   const ceil = pocketTop - hintH - gap1 - 8 - titleGlyphH / 2;
21475   const floor = titleGlyphH / 2 + 8;
21476   titleCy = Math.max(floor, Math.min(titleCy, ceil));
21477   const hintY = titleCy + titleGlyphH / 2 + gap1 + hintH / 2;
21478   return {
21479     titleH,
21480     title: { cx: 375, cy: titleCy },
21481     titleGlyphH,
21482     hintY,
21483     pocket: {

```

```

21484     y: pocketY,
21485     w: 156,
21486     h: pocketH,
21487     cxs: [145, 375, 605],
21488   },
21489   barBottom: titleH,
21490 };
21491 }
21492 // ===== FILE: src/ui/paint.ts =====
21493 /**
21494  * 切片统一画法。
21495  * 角色站在地面上，UI 用暗底金边；禁止方框套圆框、色块套立绘。
21496  */
21497 import * as PIXI from 'pixi.js';
21498 export const GOLD = 0xc9a46a;
21499 export const INK = 0x0c0e14;
21500 export const PLATE = 0x14161f;
21501 /** 村民一人一色，暖色系。外星人用冷色，两边一眼分得开 */
21502 const VILLAGER_COLOR: Readonly<Record<string, number>> = {
21503   tiezhu: 0xc4703a,
21504   dachui: 0xd9a13b,
21505   laoli: 0xb4553f,
21506   erjiu: 0xa8823f,
21507   sanshen: 0xd4736b,
21508   laoyanqiang: 0x8f7a4a,
21509 };
21510 export function label(size: number, color = 0xffffffff, bold = false): PIXI.Text {
21511   return new PIXI.Text("", {
21512     fontFamily: 'sans-serif',
21513     fontSize: size,
21514     fontWeight: bold ? 'bold' : 'normal',
21515     fill: color,
21516   });
21517 }
21518 /** 锈金漆字。门楣上的关卡名、资源数走这一套，不要裸系统字。 */
21519 export function painted(size: number, color: number, rim = '#1a1008', thick = 4): PIXI.Text {
21520   const t = label(size, color, true);
21521   t.style.stroke = rim;
21522   t.style.strokeThickness = thick;
21523   return t;
21524 }
21525 export function plate(
21526   g: PIXI.Graphics,
21527   x: number,
21528   y: number,
21529   w: number,
21530   h: number,
21531   radius = 14,
21532   alpha = 0.78,
21533 ): void {

```

```

21534 g.beginFill(PLATE, alpha).drawRoundedRect(x, y, w, h, radius).endFill();
21535 g.lineStyle(1.5, GOLD, 0.4).drawRoundedRect(x, y, w, h, radius).lineStyle(0);
21536 }
21537 export function goldBtn(g: PIXI.Graphics, x: number, y: number, w: number, h: number): void {
21538 g.beginFill(0x2a2218, 0.94).drawRoundedRect(x, y, w, h, 16).endFill();
21539 g.lineStyle(2, GOLD, 0.85).drawRoundedRect(x, y, w, h, 16).lineStyle(0);
21540 }
21541 export function rivet(g: PIXI.Graphics, x: number, y: number, r = 5): void {
21542 oldNail(g, x, y, r);
21543 }
21544 function hash01(n: number): number {
21545 const x = Math.sin(n * 127.1 + 311.7) * 43758.5453;
21546 return x - Math.floor(x);
21547 }
21548 /** 铜锈钉头：褐铜底、橙锈边、偶尔一点铜绿 */
21549 export function oldNail(g: PIXI.Graphics, x: number, y: number, r = 8, seed = 1): void {
21550 const rust = hash01(seed + 3);
21551 g.beginFill(0x0a0604, 0.42).drawCircle(x + 1.3, y + 2.3, r).endFill();
21552 g.beginFill(0x3a2214).drawCircle(x, y, r).endFill();
21553 g.beginFill(rust > 0.62 ? 0x4e8c7a : 0xc45a22, rust > 0.62 ? 0.42 : 0.55)
21554 .drawEllipse(x + r * 0.12, y + r * 0.18, r * 0.82, r * 0.7)
21555 .endFill();
21556 g.beginFill(0x5a3a22).drawCircle(x - r * 0.1, y - r * 0.16, r * 0.58).endFill();
21557 g.beginFill(0xe8c080, 0.42).drawCircle(x - r * 0.3, y - r * 0.34, r * 0.2).endFill();
21558 }
21559 /**
21560 * 顶边钉子。两头簇着钉，中间疏密不一，高低错开，不要一条尺上的点。
21561 */
21562 export function nailRow(g: PIXI.Graphics, x: number, y: number, w: number, r = 8): void {
21563 const spots = [
21564 0.045, 0.078, 0.108,
21565 0.21, 0.34, 0.49, 0.61, 0.76,
21566 0.89, 0.922, 0.955,
21567 ];
21568 spots.forEach((t, i) => {
21569 const nx = x + w * t + (hash01(i + 4) - 0.5) * 16;
21570 const ny = y + (hash01(i + 17) - 0.5) * 14 + (i < 3 || i > 7 ? (hash01(i + 29) - 0.3) * 8 : 0);
21571 const rr = r * (0.72 + hash01(i + 41) * 0.55);
21572 oldNail(g, nx, ny, rr, i + 7);
21573 });
21574 }
21575 /** 角上三颗簇钉，跟着圆角落，不要排成等边三角 */
21576 export function nailCluster(g: PIXI.Graphics, cx: number, cy: number, r = 6, seed = 0): void {
21577 const offs = [
21578 [-16, 4],
21579 [2, -8],
21580 [14, 6],
21581 ];
21582 offs.forEach(([dx, dy], i) => {
21583 oldNail(

```

```
21584     g,
21585     cx + dx + (hash01(seed + i) - 0.5) * 8,
21586     cy + dy + (hash01(seed + i + 5) - 0.5) * 7,
21587     r * (0.75 + hash01(seed + i + 9) * 0.45),
21588     seed + i,
21589 );
21590 });
21591 }
21592 /** 局外贴图：整件缩进框里，脚和链子都留着，不裁。 */
21593 export function fitSprite(
21594     parent: PIXI.Container,
21595     texture: PIXI.Texture | null,
21596     cx: number,
21597     cy: number,
21598     maxW: number,
21599     maxH: number,
21600 ): PIXI.Sprite | null {
21601     if (!texture?.baseTexture.valid || texture.width <= 1) return null;
21602     const spr = new PIXI.Sprite(texture);
21603     spr.anchor.set(0.5);
21604     spr.position.set(cx, cy);
21605     spr.scale.set(Math.min(maxW / texture.width, maxH / texture.height));
21606     spr.eventMode = 'none';
21607     parent.addChild(spr);
21608     return spr;
21609 }
21610 /**
21611  * 脚钉在同一条线上。图鉴三阶要按身体定高各自传入 scale,
21612  * 不能三张图共用一个像素比例。
21613  */
21614 export function standSprite(
21615     parent: PIXI.Container,
21616     texture: PIXI.Texture | null,
21617     cx: number,
21618     feetY: number,
21619     maxW: number,
21620     maxH: number,
21621     scale?: number,
21622 ): PIXI.Sprite | null {
21623     if (!texture?.baseTexture.valid || texture.width <= 1) return null;
21624     const spr = new PIXI.Sprite(texture);
21625     spr.anchor.set(0.5, 1);
21626     spr.scale.set(scale ?? Math.min(maxW / texture.width, maxH / texture.height));
21627     spr.position.set(cx, feetY);
21628     spr.eventMode = 'none';
21629     parent.addChild(spr);
21630     return spr;
21631 }
21632 /** 木板铺满面板，多出来的边裁掉。 */
21633 /** 铜锈斑。贴图被压平了也还能看见橙锈和铜绿 */
```

```

21634 export function copperRust(
21635   g: PIXI.Graphics,
21636   x: number,
21637   y: number,
21638   w: number,
21639   h: number,
21640 ): void {
21641   const blobs: readonly { rx: number; ry: number; rw: number; rh: number; c: number; a: number }[...
21642     { rx: 0.06, ry: 0.08, rw: 0.22, rh: 0.1, c: 0xc45a22, a: 0.28 },
21643     { rx: 0.68, ry: 0.04, rw: 0.24, rh: 0.11, c: 0xb85a2a, a: 0.26 },
21644     { rx: 0.38, ry: 0.42, rw: 0.2, rh: 0.09, c: 0xd46a28, a: 0.16 },
21645     { rx: 0.12, ry: 0.62, rw: 0.18, rh: 0.14, c: 0x4e8c7a, a: 0.16 },
21646     { rx: 0.72, ry: 0.7, rw: 0.2, rh: 0.12, c: 0xc46a28, a: 0.2 },
21647     { rx: 0.48, ry: 0.12, rw: 0.12, rh: 0.07, c: 0x5aa08b, a: 0.12 },
21648     { rx: 0.22, ry: 0.28, rw: 0.14, rh: 0.08, c: 0x8a3a18, a: 0.18 },
21649   ];
21650   for (const b of blobs) {
21651     g.beginFill(b.c, b.a).drawEllipse(x + w * b.rx, y + h * b.ry, w * b.rw, h * b.rh).endFill();
21652   }
21653 }
21654 /** 锈铁板。主页顶栏和底坞用同一块料，不再混木牌和黑玻璃。 */
21655 export function ironSlab(
21656   g: PIXI.Graphics,
21657   x: number,
21658   y: number,
21659   w: number,
21660   h: number,
21661   radius = 12,
21662 ): void {
21663   g.beginFill(0x0a0604, 0.5).drawRoundedRect(x + 4, y + 6, w, h, radius).endFill();
21664   g.beginFill(0x2a1810).drawRoundedRect(x, y, w, h, radius).endFill();
21665   g.beginFill(0x3d2418, 0.72).drawRoundedRect(x + 7, y + 7, w - 14, h - 14, Math.max(4, radius - ...
21666   g.lineStyle(2, 0x6a4a28, 0.4).drawRoundedRect(x + 2, y + 2, w - 4, h - 4, radius).lineStyle(0);
21667   const inset = Math.min(18, w * 0.06, h * 0.22);
21668   rivet(g, x + inset, y + inset);
21669   rivet(g, x + w - inset, y + inset);
21670   rivet(g, x + inset, y + h - inset);
21671   rivet(g, x + w - inset, y + h - inset);
21672 }
21673 /**
21674  * 格子的地面垫。开打之后才画，布阵阶段画的是空心格（那时候格线本身就是热区）。
21675  * 只画在自家那 12 格上，上方空场仍然不铺——空场是舞台，见 BattleScene 的头注释。
21676  */
21677 export function queuePad(
21678   g: PIXI.Graphics,
21679   cx: number,
21680   feetY: number,
21681   opts: { empty: boolean; hot: boolean; front: boolean },
21682 ): void {
21683   const rx = opts.empty ? 18 : 16;

```

```
21684 const ry = opts.empty ? 7 : 6;
21685 g.beginFill(0x000000, opts.empty ? 0.22 : 0.18);
21686 g.drawEllipse(cx, feetY + 8, rx, ry);
21687 g.endFill();
21688 if (opts.hot) {
21689   g.lineStyle(3, 0x9be08a, 0.95).drawEllipse(cx, feetY + 8, rx + 4, ry + 3).lineStyle(0);
21690   g.beginFill(0x9be08a, 0.12).drawEllipse(cx, feetY + 8, rx, ry).endFill();
21691   return;
21692 }
21693 if (opts.empty) {
21694   g.lineStyle(2, GOLD, 0.55).drawEllipse(cx, feetY + 8, rx, ry).lineStyle(0);
21695   g.lineStyle(1.2, GOLD, 0.25).drawEllipse(cx, feetY + 8, rx * 0.62, ry * 0.62).lineStyle(0);
21696   return;
21697 }
21698 if (opts.front) {
21699   g.beginFill(GOLD, 0.18).drawEllipse(cx, feetY + 8, rx + 8, ry + 5).endFill();
21700   g.lineStyle(3.6, GOLD, 0.95).drawEllipse(cx, feetY + 8, rx + 7, ry + 4).lineStyle(0);
21701   g.lineStyle(1.4, 0xffe08a, 0.75).drawEllipse(cx, feetY + 8, rx + 1, ry + 1).lineStyle(0);
21702 }
21703 }
21704 /**
21705  * 够得到哪儿：贴地椭圆，不是竖条。近战是围着人的一小片，远程铺到最远点。
21706  *
21707  * 布阵阶段每个人都画一层淡的。§ 8.1 记着那个 bug —— 上一版战场太长而射程太短，
21708  * 后排两格打不到任何东西，而屏幕上完全看不出来。这一层就是为了让
21709  * 「他站这儿够不够得着」变成看一眼的事，而不是打完一局才发现。
21710  */
21711 export function rangeArea(
21712   g: PIXI.Graphics,
21713   cx: number,
21714   feetY: number,
21715   reachY: number,
21716   color: number,
21717   melee: boolean,
21718 ): void {
21719   const forward = Math.max(40, Math.abs(feetY - reachY));
21720   if (melee) {
21721     const rx = Math.max(78, forward * 1.2);
21722     const ry = Math.max(52, forward * 0.9);
21723     const cy = feetY - ry * 0.28;
21724     g.beginFill(color, 0.18).drawEllipse(cx, cy, rx, ry).endFill();
21725     g.lineStyle(2.6, color, 0.78).drawEllipse(cx, cy, rx, ry).lineStyle(0);
21726     g.lineStyle(1.2, 0xffffffff, 0.32).drawEllipse(cx, cy, rx * 0.9, ry * 0.9).lineStyle(0);
21727     return;
21728   }
21729   const ry = forward * 0.52;
21730   const rx = Math.min(300, Math.max(120, forward * 0.4));
21731   const cy = (feetY + reachY) / 2;
21732   g.beginFill(color, 0.1).drawEllipse(cx, cy, rx, ry).endFill();
21733   g.lineStyle(2.2, color, 0.55).drawEllipse(cx, cy, rx, ry).lineStyle(0);
```

```
21734 }
21735 /**
21736 * 经验条。玩家唯一能看见的「还差多久捡下一件破烂」。
21737 *
21738 * 攒满了不画成空条：破烂发完之后这条还在，得让它看起来是完成态而不是坏了。
21739 */
21740 export function expBar(
21741   g: PIXI.Graphics,
21742   x: number,
21743   y: number,
21744   width: number,
21745   ratio: number,
21746   done: boolean,
21747 ): void {
21748   const h = 12;
21749   const r = 6;
21750   g.beginFill(0x000000, 0.5).drawRoundedRect(x, y, width, h, r).endFill();
21751   const w = done ? width : Math.max(0, Math.min(1, ratio)) * width;
21752   if (w > 0) {
21753     g.beginFill(done ? 0x9be08a : GOLD, 0.95).drawRoundedRect(x, y, w, h, r).endFill();
21754   }
21755   g.lineStyle(1.2, GOLD, 0.45).drawRoundedRect(x, y, width, h, r).lineStyle(0);
21756 }
21757 /** 铺满一块矩形。顶栏锈牌、出村铁门要拉到设计尺寸，不能按原图比例缩成小方块。 */
21758 export function fillSprite(
21759   parent: PIXI.Container,
21760   texture: PIXI.Texture | null,
21761   cx: number,
21762   cy: number,
21763   w: number,
21764   h: number,
21765 ): PIXI.Sprite | null {
21766   if (!texture?.baseTexture.valid || texture.width <= 1) return null;
21767   const spr = new PIXI.Sprite(texture);
21768   spr.anchor.set(0.5);
21769   spr.position.set(cx, cy);
21770   spr.scale.set(w / texture.width, h / texture.height);
21771   spr.eventMode = 'none';
21772   parent.addChild(spr);
21773   return spr;
21774 }
21775 /** 钉在泥地里的木牌。关卡桩、能升阶木牌的兜底，贴图没来时也能认。 */
21776 export function woodPlank(
21777   g: PIXI.Graphics,
21778   x: number,
21779   y: number,
21780   w: number,
21781   h: number,
21782   radius = 8,
21783 ): void {
```

```

21784 g.beginFill(0x1a1008, 0.45).drawRoundedRect(x + 3, y + 5, w, h, radius).endFill();
21785 g.beginFill(0x5a3a18).drawRoundedRect(x, y, w, h, radius).endFill();
21786 g.beginFill(0x6e4a22, 0.78).drawRoundedRect(x + 5, y + 4, w - 10, h - 8, Math.max(3, radius - 3...
21787 g.strokeStyle(2, 0x3a2410, 0.55).drawRoundedRect(x + 1, y + 1, w - 2, h - 2, radius).lineStyle(0);
21788 }
21789 export function hpBar(
21790   g: PIXI.Graphics,
21791   cx: number,
21792   y: number,
21793   width: number,
21794   ratio: number,
21795   color: number,
21796   barH = 6,
21797 ): void {
21798   const w = Math.max(0, Math.min(1, ratio)) * width;
21799   const r = Math.max(1, Math.round(barH / 2));
21800   g.beginFill(0x000000, 0.55).drawRoundedRect(cx - width / 2, y, width, barH, r).endFill();
21801   if (w > 0) g.beginFill(color, 0.95).drawRoundedRect(cx - width / 2, y, w, barH, r).endFill();
21802 }
21803 // ===== FILE: src/utils/__tests__/safeDestroy.test.ts =====
21804 import { describe, expect, it, vi } from 'vitest';
21805 import { safeDestroy } from '@utils/safeDestroy';
21806 describe('safeDestroy', () => {
21807   it('destroyed 的不再调 destroy', () => {
21808     const destroy = vi.fn();
21809     safeDestroy({ destroyed: true, destroy } as { destroyed: boolean });
21810     expect(destroy).not.toHaveBeenCalled();
21811   });
21812   it('二次 destroy 抛错也不往外冒', () => {
21813     const obj = {
21814       destroyed: false,
21815       destroy() {
21816         if (this.destroyed) throw new Error("Cannot read properties of null (reading 'refCount')");
21817         this.destroyed = true;
21818       },
21819     };
21820     safeDestroy(obj);
21821     safeDestroy(obj);
21822     expect(obj.destroyed).toBe(true);
21823   });
21824 });
21825 // ===== FILE: src/utils/bindPointerTap.ts =====
21826 /**
21827  * 可点击绑定（见 @/minigame/index.ts 统一交互层）：
21828  * - 微信小游戏：canvasTapRouter（touchstart/touchend + 设计坐标 hitTest）
21829  * - 纯浏览器：pointerdown + pointerup
21830  * 可选 onLongPress：按住约 450ms 触发，松手不再走短按。
21831  */
21832 import * as PIXI from 'pixi.js';
21833 import { Platform } from '@core/PlatformService';

```

```
21834 import { playSfx } from '@core/SfxPlayer';
21835 import { registerCanvasTap } from './canvasTapRouter';
21836 import { clientEventToDesign } from './clientEventToDesign';
21837 import { deferAfterPointerEvent } from './deferAfterPointer';
21838 const DEFAULT_LONG_PRESS_MS = 450;
21839 const HOLD_SLOP = 14;
21840 export function bindPointerTap(
21841   target: PIXI.Container,
21842   /** 参数是本次 tap 的设计坐标；只关心「点了」的调用方直接忽略即可。 */
21843   fn: (dx: number, dy: number) => void,
21844   opts?: {
21845     guard?: () => boolean;
21846     blockTap?: () => boolean;
21847     pointGuard?: (dx: number, dy: number) => boolean;
21848     sync?: boolean;
21849     /** 不播点击音（长按菜单、全屏挡板） */
21850     silent?: boolean;
21851     onLongPress?: () => void;
21852     longPressMs?: number;
21853   },
21854 ): void {
21855   let fired = false;
21856   const run = (dx: number, dy: number): void => {
21857     if (fired) return;
21858     if (opts?.guard && !opts.guard()) return;
21859     if (opts?.blockTap?.()) return;
21860     fired = true;
21861     if (opts?.sync) {
21862       try { fn(dx, dy); } catch (err) { console.error('[bindPointerTap sync]', err); }
21863       fired = false;
21864       return;
21865     }
21866     deferAfterPointerEvent(() => {
21867       fired = false;
21868       fn(dx, dy);
21869     });
21870   };
21871   if (Platform.isMinigame) {
21872     if (target.eventMode === 'none' || !target.eventMode) {
21873       target.eventMode = 'static';
21874     }
21875     registerCanvasTap({
21876       target,
21877       fn: run,
21878       guard: opts?.guard,
21879       blockTap: opts?.blockTap,
21880       pointGuard: opts?.pointGuard,
21881       sync: opts?.sync,
21882       silent: opts?.silent,
21883       onLongPress: opts?.onLongPress,
```

```
21884     longPressMs: opts?.longPressMs,
21885   });
21886   return;
21887 }
21888 target.eventMode = target.eventMode === 'none' ? 'static' : (target.eventMode || 'static');
21889 let armed = false;
21890 let longPressed = false;
21891 let holdTimer: ReturnType<typeof setTimeout> | null = null;
21892 let startX = 0;
21893 let startY = 0;
21894 const clearHold = (): void => {
21895   if (holdTimer !== null) {
21896     clearTimeout(holdTimer);
21897     holdTimer = null;
21898   }
21899 };
21900 target.on('pointerdown', (e: PIXI.FederatedPointerEvent) => {
21901   armed = true;
21902   longPressed = false;
21903   clearHold();
21904   startX = e.global.x;
21905   startY = e.global.y;
21906   if (!opts?.onLongPress) return;
21907   const ms = opts.longPressMs ?? DEFAULT_LONG_PRESS_MS;
21908   holdTimer = setTimeout(() => {
21909     holdTimer = null;
21910     if (!armed) return;
21911     if (opts.guard && !opts.guard()) return;
21912     if (opts.blockTap?.()) return;
21913     longPressed = true;
21914     try {
21915       opts.onLongPress!();
21916     } catch (err) {
21917       console.error('[bindPointerTap longPress]', err);
21918     }
21919   }, ms);
21920 });
21921 target.on('pointermove', (e: PIXI.FederatedPointerEvent) => {
21922   if (!armed || longPressed || holdTimer === null) return;
21923   const dx = e.global.x - startX;
21924   const dy = e.global.y - startY;
21925   if (dx * dx + dy * dy > HOLD_SLOP * HOLD_SLOP) clearHold();
21926 });
21927 target.on('pointerup', (e: PIXI.FederatedPointerEvent) => {
21928   clearHold();
21929   if (!armed) return;
21930   armed = false;
21931   if (longPressed) return;
21932   const p = e.nativeEvent ? clientEventToDesign(e.nativeEvent) : { x: e.global.x, y: e.global.y };
21933   if (!opts?.silent) playSfx('ui_tap', 40);
```

```

21934     run(p.x, p.y);
21935   });
21936   target.on('pointerupoutside', () => {
21937     clearHold();
21938     armed = false;
21939   });
21940   target.on('pointercancel', () => {
21941     clearHold();
21942     armed = false;
21943   });
21944 }
21945 // ===== FILE: src/utils/canvasTapRouter.ts =====
21946 /**
21947  * 微信真机: tap 走 canvas touchstart/touchend + 设计坐标 hitTest (勿混 pointerdown, 会覆盖 _active)。
21948  * 可选长按: 按住达阈值且位移未超 slop 时触发 onLongPress, 松手不再发 tap。
21949  */
21950 import { Platform } from '@core/PlatformService';
21951 import { playSfx } from '@core/SfxPlayer';
21952 import { clientEventToDesign } from './clientEventToDesign';
21953 import { containsDesignPoint, isOnStage, pickTopmostHit } from './hitTestDesign';
21954 import { deferAfterPointerEvent } from './deferAfterPointer';
21955 import { getTouchCanvas } from './touchCanvas';
21956 const TAP_SLOP = 14;
21957 const DEFAULT_LONG_PRESS_MS = 450;
21958 interface TapBinding {
21959   target: import('pixi.js').Container;
21960   /** 收到本次 tap 的设计坐标。整块热区内部再分区 (编队台选人) 要用。 */
21961   fn: (dx: number, dy: number) => void;
21962   guard?: () => boolean;
21963   blockTap?: () => boolean;
21964   /** 设计坐标额外过滤 (如滚动列表视口外不响应) */
21965   pointGuard?: (dx: number, dy: number) => boolean;
21966   /** 为 true 时在 touchend 内同步执行 (tt.addShortcut 等必须用户手势同步调用的 API) */
21967   sync?: boolean;
21968   silent?: boolean;
21969   onLongPress?: () => void;
21970   longPressMs?: number;
21971 }
21972 let _installed = false;
21973 let _bindings: TapBinding[] = [];
21974 let _active: { binding: TapBinding; x: number; y: number } | null = null;
21975 let _holdTimer: ReturnType<typeof setTimeout> | null = null;
21976 let _longPressed = false;
21977 let _onStart: EventListener | null = null;
21978 let _onMove: EventListener | null = null;
21979 let _onEnd: EventListener | null = null;
21980 function clearHoldTimer(): void {
21981   if (_holdTimer != null) {
21982     clearTimeout(_holdTimer);
21983     _holdTimer = null;

```

```

21984 }
21985 }
21986 function pickBinding(dx: number, dy: number): TapBinding | null {
21987 // 连 target.parent 都不看够：切场景后旧树整棵脱离舞台，parent 还在，会一直堆着。
21988 _bindings = _bindings.filter((b) => b.target.parent && isOnStage(b.target));
21989 // 禁用态也要参与 hitTest，否则会穿透点到下层按钮（抽卡结果页点「确定」误触底层十连）
21990 const hits = _bindings.filter((b) =>
21991   containsDesignPoint(b.target, dx, dy)
21992   && (!b.pointGuard || b.pointGuard(dx, dy)));
21993 if (!hits.length) return null;
21994 const top = pickTopmostHit(hits.map((b) => b.target), dx, dy);
21995 if (!top) return null;
21996 return hits.find((b) => b.target === top) ?? hits[hits.length - 1];
21997 }
21998 function armLongPress(binding: TapBinding): void {
21999   clearHoldTimer();
22000   if (!binding.onLongPress) return;
22001   const ms = binding.longPressMs ?? DEFAULT_LONG_PRESS_MS;
22002   _holdTimer = setTimeout(() => {
22003     _holdTimer = null;
22004     const act = _active;
22005     if (!act || act.binding !== binding) return;
22006     if (!binding.target.parent) return;
22007     if (binding.guard && !binding.guard()) return;
22008     if (binding.blockTap?.()) return;
22009     _longPressed = true;
22010     try {
22011       binding.onLongPress!();
22012     } catch (err) {
22013       console.error('[canvasTapRouter longPress]', err);
22014     }
22015   }, ms);
22016 }
22017 function ensureInstalled(): void {
22018   if (_installed || !Platform.isMinigame) return;
22019   const canvas = getTouchCanvas();
22020   if (!canvas?.addEventListener) return;
22021   _onStart = ((e: Event) => {
22022     clearHoldTimer();
22023     _longPressed = false;
22024     const p = clientEventToDesign(e);
22025     const binding = pickBinding(p.x, p.y);
22026     _active = binding ? { binding, x: p.x, y: p.y } : null;
22027     if (binding) armLongPress(binding);
22028   }) as EventListener;
22029   _onMove = ((e: Event) => {
22030     const act = _active;
22031     if (!act || !_longPressed) return;
22032     const p = clientEventToDesign(e);
22033     const dx = p.x - act.x;

```

```

22034   const dy = p.y - act.y;
22035   if (dx * dx + dy * dy > TAP_SLOP * TAP_SLOP) {
22036     clearHoldTimer();
22037   }
22038 }) as EventListener;
22039 _onEnd = ((e: Event) => {
22040   clearHoldTimer();
22041   const act = _active;
22042   const wasLong = _longPressed;
22043   _active = null;
22044   _longPressed = false;
22045   if (!act || wasLong) return;
22046   const p = clientEventToDesign(e);
22047   const movedX = p.x - act.x;
22048   const movedY = p.y - act.y;
22049   if (movedX * movedX + movedY * movedY > TAP_SLOP * TAP_SLOP) return;
22050   // 松手处再取一次最上层。选中后立绘放大，起点热区还钉在旧人身上。
22051   const fresh = pickBinding(p.x, p.y) ?? act.binding;
22052   const b = fresh.target.parent ? fresh : act.binding;
22053   if (!b.target.parent) return;
22054   if (b.guard && !b.guard()) return;
22055   if (b.blockTap?.()) return;
22056   // 必须在 touchend 里同步播。丢进 setTimeout 微信当非手势，直接静音。
22057   if (!b.silent) playSfx('ui_tap', 40);
22058   if (b.sync) {
22059     try { b.fn(p.x, p.y); } catch (err) { console.error('[canvasTapRouter sync]', err); }
22060   } else {
22061     deferAfterPointerEvent(() => b.fn(p.x, p.y));
22062   }
22063 }) as EventListener;
22064 const onCancel = (): void => {
22065   clearHoldTimer();
22066   _active = null;
22067   _longPressed = false;
22068 };
22069 canvas.addEventListener('touchstart', _onStart, { passive: true });
22070 canvas.addEventListener('touchmove', _onMove, { passive: true });
22071 canvas.addEventListener('touchend', _onEnd);
22072 canvas.addEventListener('touchcancel', onCancel);
22073 _installed = true;
22074 }
22075 export function registerCanvasTap(binding: TapBinding): () => void {
22076   ensureInstalled();
22077   _bindings.push(binding);
22078   return () => {
22079     const i = _bindings.indexOf(binding);
22080     if (i !== -1) _bindings.splice(i, 1);
22081   };
22082 }
22083 // ===== FILE: src/utils/clientEventToDesign.ts =====

```

```
22084 /**
22085  * 微信 touch / pointer → 设计坐标（750 宽）。
22086  * 统一规则见 @/minigame/index.ts；对齐 game2D_huahua BoardView._rawToLocal。
22087  */
22088 import * as PIXI from 'pixi.js';
22089 import { Game } from '@core/Game';
22090 import { designPointToContainerLocal } from './hitTestDesign';
22091 export function clientEventToDesign(e: unknown): { x: number; y: number } {
22092   const ev = e as {
22093     clientX?: number;
22094     clientY?: number;
22095     x?: number;
22096     y?: number;
22097     touches?: Array<{ clientX?: number; clientY?: number }>;
22098     changedTouches?: Array<{ clientX?: number; clientY?: number }>;
22099   };
22100   const t0 = ev.changedTouches?.[0] ?? ev.touches?.[0];
22101   const cx = ev.clientX ?? t0?.clientX ?? ev.x ?? 0;
22102   const cy = ev.clientY ?? t0?.clientY ?? ev.y ?? 0;
22103   // 对齐 game2D_huahua: clientX(逻辑像素) → 设计坐标，单一比例 designWidth/screenWidth。
22104   // 不经 EventSystem patch / toLocal，避免依赖 worldTransform 是否最新。
22105   const ratio = Game.designWidth / Game.screenWidth;
22106   return { x: cx * ratio, y: cy * ratio };
22107 }
22108 export function designPointToLocal(target: PIXI.Container, dx: number, dy: number): PIXI.Point {
22109   return designPointToContainerLocal(target, dx, dy);
22110 }
22111 /** touch/pointer 事件 → 某容器本地设计坐标（棋盘拖拽 / hitTest 共用） */
22112 export function designEventToLocal(target: PIXI.Container, e: unknown): { x: number; y: number } {
22113   const design = clientEventToDesign(e);
22114   const local = designPointToLocal(target, design.x, design.y);
22115   return { x: local.x, y: local.y };
22116 }
22117 // ===== FILE: src/utils/deferAfterPointer.ts =====
22118 /**
22119  * 推迟到当前指针事件派发完成后再执行。
22120  * 避免 pointertap 内切场景/destroy 节点后，Pixi pointerup 仍遍历 pressTargets 报 null.scale。
22121  */
22122 import { Game } from '@core/Game';
22123 import { Platform } from '@core/PlatformService';
22124 let _rafId = 0;
22125 const _queue: Array<() => void> = [];
22126 export function deferAfterPointerEvent(fn: () => void): void {
22127   _queue.push(fn);
22128   if (_rafId !== 0) return;
22129   const flush = () => {
22130     _rafId = 0;
22131     const batch = _queue.splice(0);
22132     for (const run of batch) {
22133       try {
```

```

22134     run();
22135   } catch (err) {
22136     console.error('[deferAfterPointer]', err);
22137   }
22138 }
22139 };
22140 _rafId = -1;
22141 // 开发者工具也走 setTimeout: 微信模拟器里 ticker.addOnce 会偶发丢,
22142 // 点下去像没反应。真机同样不能靠 addOnce (见 deferNextFrame 注释)。
22143 if (Platform.isMinigame) {
22144   setTimeout(flush, 0);
22145   return;
22146 }
22147 if (Game.ticker?.started) {
22148   Game.ticker.addOnce(flush);
22149   return;
22150 }
22151 _rafId = requestAnimationFrame(flush);
22152 }
22153 /** 推迟到下一帧 (场景 switchTo 后的 build 专用, 避免与 Pixi 挂载同帧冲突) 。 */
22154 export function deferNextFrame(fn: () => void): void {
22155   const run = () => {
22156     try {
22157       fn();
22158     } catch (err) {
22159       console.error('[deferNextFrame]', err);
22160     }
22161   };
22162   // 真机 pointerup→setTimeout(switchTo) 后 ticker.addOnce 常不触发, build 永远不执行 → 黑屏
22163   if (Platform.isMinigame && !Platform.isDevtools) {
22164     setTimeout(run, 0);
22165     return;
22166   }
22167   if (Game.ticker?.started) {
22168     Game.ticker.addOnce(run);
22169     return;
22170   }
22171   requestAnimationFrame(run);
22172 }
22173 // ===== FILE: src/utils/hitTestDesign.ts =====
22174 /**
22175  * 设计坐标 hitTest (750 布局), 供 canvasTapRouter / ScrollList / 编队列表共用。
22176  *
22177  * 注意: 小游戏 tap 不走 Pixi 原生 hitTest, 必须在此自行尊重祖先 mask。
22178  * 否则滚动列表视觉上被裁切后, 仍会抢走顶栏「返回」等按钮的点击。
22179  */
22180 import * as PIXI from 'pixi.js';
22181 import { Game } from '@core/Game';
22182 /** 设计坐标 → 容器本地 (含父级 scale/pivot, 地图 cover 缩放后点击才准) */
22183 export function designPointToContainerLocal(target: PIXI.Container, dx: number, dy: number): PIXI...

```

```

22184     return target.toLocal(new PIXI.Point(dx, dy), Game.stage);
22185 }
22186 function localContains(
22187     local: PIXI.Point,
22188     ha: PIXI.IHitArea | null | undefined,
22189     fallback: PIXI.Container,
22190 ): boolean {
22191     if (ha instanceof PIXI.Rectangle) {
22192         return local.x >= ha.x && local.x <= ha.x + ha.width
22193             && local.y >= ha.y && local.y <= ha.y + ha.height;
22194     }
22195     if (ha instanceof PIXI.Circle) {
22196         const dx1 = local.x - ha.x;
22197         const dy1 = local.y - ha.y;
22198         return dx1 * dx1 + dy1 * dy1 <= ha.radius * ha.radius;
22199     }
22200     if (ha instanceof PIXI.RoundedRectangle) {
22201         return ha.contains(local.x, local.y);
22202     }
22203     if (ha && typeof (ha as { contains?: unknown }).contains === 'function') {
22204         return (ha as { contains: (x: number, y: number) => boolean }).contains(local.x, local.y);
22205     }
22206     const b = fallback.getLocalBounds();
22207     return local.x >= b.x && local.x <= b.x + b.width
22208         && local.y >= b.y && local.y <= b.y + b.height;
22209 }
22210 /**
22211  * 祖先链上若有 mask，设计点必须落在 mask 形状内（对齐 Pixi 视觉裁切）。
22212  * 滚动列表用 Graphics.drawRect 做视口 mask 时，滚出视口的卡片不再可点。
22213  */
22214 /** mask 可能是容器本身，也可能是包了一层的 MaskData */
22215 function resolveMaskObject(mask: PIXI.Container | PIXI.MaskData | null): PIXI.Container | null {
22216     if (!mask) return null;
22217     const obj = mask instanceof PIXI.MaskData ? mask.maskObject : mask;
22218     return (obj as PIXI.Container | null) ?? null;
22219 }
22220 function insideAncestorMasks(target: PIXI.Container, dx: number, dy: number): boolean {
22221     let cur: PIXI.Container | null = target;
22222     while (cur) {
22223         const maskObj = resolveMaskObject(cur.mask);
22224         if (maskObj) {
22225             const local = designPointToContainerLocal(maskObj, dx, dy);
22226             if (!localContains(local, maskObj.hitArea, maskObj)) return false;
22227         }
22228         cur = cur.parent;
22229     }
22230     return true;
22231 }
22232 /**
22233  * 节点是否还挂在当前舞台上。

```

```

22234 *
22235 * 切场景只做 stage.removeChild(旧场景.container)，旧场景内部的节点 parent 仍在、
22236 * visible 仍是 true，Pixi 的 worldVisible 也照样返回 true——它只看 visible 标志，
22237 * 不管这棵树有没有连到舞台。于是上一局战斗的选卡热区会继续在村口抢点击，
22238 * 把村口点人的整块热区压住（点人无反应、只有没被盖住的那个能选）。
22239 */
22240 export function isOnStage(target: PIXI.Container): boolean {
22241     const stage = Game.stage;
22242     if (!stage) return true;
22243     let cur: PIXI.Container = target;
22244     while (cur.parent) cur = cur.parent as PIXI.Container;
22245     return cur === stage;
22246 }
22247 export function containsDesignPoint(target: PIXI.Container, dx: number, dy: number): boolean {
22248     if (!target.parent || !target.visible || target.worldVisible === false) return false;
22249     if (target.eventMode === 'none') return false;
22250     if (!isOnStage(target)) return false;
22251     if (!insideAncestorMasks(target, dx, dy)) return false;
22252     const local = designPointToContainerLocal(target, dx, dy);
22253     return localContains(local, target.hitArea, target);
22254 }
22255 /** 从候选容器里取最上层命中项（对齐 Pixi 绘制顺序：后绘制的在上） */
22256 export function pickTopmostHit(
22257     candidates: PIXI.Container[],
22258     dx: number,
22259     dy: number,
22260 ): PIXI.Container | null {
22261     let best: PIXI.Container | null = null;
22262     let bestPath: number[] | null = null;
22263     for (const target of candidates) {
22264         if (!containsDesignPoint(target, dx, dy)) continue;
22265         const path = worldPath(target);
22266         if (!bestPath || comparePath(path, bestPath) >= 0) {
22267             best = target;
22268             bestPath = path;
22269         }
22270     }
22271     return best;
22272 }
22273 /**
22274 * 从根到叶的子节点索引链。
22275 *
22276 * 必须保留成数组逐层比，不能压成一个数。曾经压成 `order * 1000 + index` 从叶往根
22277 * 累加，等于把叶节点的索引放到了最高位：Overlay 层的 GM 按钮算出 3002，战斗场景里
22278 * 埋得更深的点怪热区算出 15001，热区反而赢了——点 GM 按钮弹出的是怪物说明。
22279 * 只要下层子树够深，它的叶节点就能压过顶层容器，「Overlay 永远在最上层」这个前提
22280 * 就被算法本身破坏了。
22281 */
22282 function worldPath(target: PIXI.Container): number[] {
22283     const path: number[] = [];

```

```

22284 let cur: PIXI.Container | null = target;
22285 while (cur?.parent) {
22286   path.push(cur.parent.getChildIndex(cur));
22287   cur = cur.parent;
22288 }
22289 return path.reverse();
22290 }
22291 /** 顶层先分胜负，同层再往下比；祖先相同时更深的那个在上 */
22292 function comparePath(a: readonly number[], b: readonly number[]): number {
22293   const n = Math.max(a.length, b.length);
22294   for (let i = 0; i < n; i++) {
22295     const av = a[i] ?? -1;
22296     const bv = b[i] ?? -1;
22297     if (av !== bv) return av - bv;
22298   }
22299   return 0;
22300 }
22301 // ===== FILE: src/utils/safeDestroy.ts =====
22302 /**
22303  * PIXI Graphics.destroy 会先把 _geometry 置空，再标 destroyed。
22304  * 二次调用读 null.refCount，微信小程序直接 MiniProgramError。
22305  */
22306 export function safeDestroy(obj: { destroyed?: boolean } | null | undefined, options?: unknown): ...
22307 if (!obj || obj.destroyed) return;
22308 const destroy = (obj as { destroy?: (opts?: unknown) => void }).destroy;
22309 if (typeof destroy !== 'function') return;
22310 try {
22311   destroy.call(obj, options);
22312 } catch {
22313   /* 已经拆过，或父节点带 children 拆过 */
22314 }
22315 }
22316 // ===== FILE: src/utils/touchCanvas.ts =====
22317 /**
22318  * 微信双 canvas: TouchEvent 桥接挂在 GameGlobal.canvas (pixi-primary) 。
22319  */
22320 import { Game } from '@core/Game';
22321 declare const GameGlobal: { canvas?: TouchCanvasLike } | undefined;
22322 export type TouchCanvasLike = {
22323   addEventListener: (type: string, handler: EventListener, options?: boolean | AddEventListenerOp...
22324   removeEventListener: (type: string, handler: EventListener, options?: boolean | AddEventListene...
22325 };
22326 export function getTouchCanvas(): TouchCanvasLike {
22327   const bridged = GameGlobal?.canvas;
22328   if (bridged && typeof bridged.addEventListener === 'function') {
22329     return bridged;
22330   }
22331   return Game.app.view as unknown as TouchCanvasLike;
22332 }
22333 // ===== FILE: cloudfunctions/cunkou-api/index.js =====

```

```
22334 const { handleLogin } = require('./lib/auth');
22335 const { handlePull, handlePush } = require('./lib/save');
22336 const { respond, parseEvent, preflight } = require('./lib/http');
22337 const { getGameKey, getScopedGameKey } = require('./lib/config');
22338 const ROUTES = {
22339   'GET /health': async () => ({
22340     ok: true,
22341     gameKey: getGameKey(),
22342     scopedGameKeys: { wx: getScopedGameKey('wx'), dy: getScopedGameKey('dy'), tap: getScopedGameK...
22343     ts: Date.now(),
22344   }),
22345   'POST /health': async () => ({
22346     ok: true,
22347     gameKey: getGameKey(),
22348     scopedGameKeys: { wx: getScopedGameKey('wx'), dy: getScopedGameKey('dy'), tap: getScopedGameK...
22349     ts: Date.now(),
22350   }),
22351   'POST /login': handleLogin,
22352   'POST /save/pull': handlePull,
22353   'POST /save/push': handlePush,
22354 };
22355 exports.main = async (event, context) => {
22356   try {
22357     if (event && event.httpMethod === 'OPTIONS') {
22358       return preflight();
22359     }
22360     const req = parseEvent(event);
22361     const key = `${req.method} ${req.path}`;
22362     const handler = ROUTES[key];
22363     if (!handler) {
22364       return respond(404, { ok: false, code: 'NOT_FOUND', error: `no route: ${key}` });
22365     }
22366     const result = await handler(req, context);
22367     if (result && typeof result === 'object' && 'statusCode' in result) {
22368       return result;
22369     }
22370     return respond(200, { ok: true, data: result });
22371   } catch (error) {
22372     const code = error && error.code ? error.code : 'INTERNAL';
22373     const status = error && error.status ? error.status : 500;
22374     const message = (error && error.message) || String(error);
22375     console.error('[cunkou-api] error:', code, message, error && error.stack);
22376     const out = { ok: false, code, error: message };
22377     if (error && error.data !== undefined) {
22378       out.data = error.data;
22379     }
22380     return respond(status, out);
22381   }
22382 };
22383 // ===== FILE: cloudfunctions/cunkou-api/lib/auth.js =====
```

```
22384 const jwt = require('jsonwebtoken');
22385 const { httpError } = require('./http');
22386 const {
22387   getGameKey,
22388   getScopedGameKey,
22389   gameKeyUpper,
22390   getJwtSecret: readJwtSecret,
22391   getTtlSec,
22392   getPlatformCredential,
22393 } = require('./config');
22394 const SUPPORTED_PLATFORMS = new Set(['wx', 'dy', 'tap', 'anon']);
22395 function getJwtSecret() {
22396   const secret = readJwtSecret();
22397   if (!secret) {
22398     throw httpError(500, 'NO_JWT_SECRET', `${gameKeyUpper()}_JWT_SECRET 未配置`);
22399   }
22400   return secret;
22401 }
22402 async function handleLogin(req) {
22403   const body = req.body || {};
22404   const platform = String(body.platform || '').toLowerCase();
22405   if (!SUPPORTED_PLATFORMS.has(platform)) {
22406     throw httpError(400, 'BAD_PLATFORM', `unsupported platform: ${platform}`);
22407   }
22408   let platformUid = '';
22409   if (platform === 'wx') {
22410     const wxSession = await wxCode2Session(body.code);
22411     platformUid = wxSession.openid;
22412   } else if (platform === 'dy') {
22413     platformUid = await ttCode2Openid(body.code);
22414   } else if (platform === 'tap') {
22415     const tapSession = await tapCode2Session(body.code);
22416     platformUid = tapSession.openid;
22417   } else if (platform === 'anon') {
22418     const id = String(body.anonId || '').trim();
22419     if (!id) throw httpError(400, 'NO_ANON_ID', 'anonId 缺失');
22420     if (! /^[A-Za-z0-9_-\.:]{8,128}$/.test(id)) {
22421       throw httpError(400, 'BAD_ANON_ID', 'anonId 非法');
22422     }
22423     platformUid = id;
22424   }
22425   const userId = `${platform}:${platformUid}`;
22426   const ttlSec = getTtlSec();
22427   const scopedGameKey = getScopedGameKey(platform);
22428   const now = Math.floor(Date.now() / 1000);
22429   const token = jwt.sign(
22430     { sub: userId, plt: platform, gk: scopedGameKey, iat: now },
22431     getJwtSecret(),
22432     { expiresIn: ttlSec },
22433   );
```

```
22434 return {
22435   token,
22436   userId,
22437   platform,
22438   gameKey: scopedGameKey,
22439   baseGameKey: getGameKey(),
22440   expiresAt: (now + ttlSec) * 1000,
22441   ttlSec,
22442 };
22443 }
22444 function requireUser(req) {
22445   const authHeader = (req.headers && req.headers.authorization) || '';
22446   const match = /^Bearer\s+(.+)$/i.exec(authHeader);
22447   if (!match) throw httpError(401, 'NO_TOKEN', '缺少 Authorization: Bearer <token>');
22448   let payload;
22449   try {
22450     payload = jwt.verify(match[1].trim(), getJwtSecret());
22451   } catch (error) {
22452     throw httpError(401, 'BAD_TOKEN', error && error.message ? error.message : 'token 无效');
22453   }
22454   const userId = payload && payload.sub;
22455   if (!userId || typeof userId !== 'string' || !userId.includes(':')) {
22456     throw httpError(401, 'BAD_TOKEN', 'token sub 非法');
22457   }
22458   const platform = payload.plt || userId.split(':')[0];
22459   const currentGameKey = getScopedGameKey(platform);
22460   if (payload.gk && payload.gk !== currentGameKey) {
22461     throw httpError(
22462       401,
22463       'BAD_TOKEN',
22464       `token gameKey=${payload.gk} 与当前 scoped GAME_KEY=${currentGameKey} 不匹配`,
22465     );
22466   }
22467   return { userId, platform, gameKey: currentGameKey };
22468 }
22469 async function wxCode2Session(code) {
22470   const appid = getPlatformCredential('wx', 'APPID');
22471   const secret = getPlatformCredential('wx', 'SECRET');
22472   if (!appid || !secret) throw httpError(500, 'NO_WX_CFG', `${gameKeyUpper()}_WX_APPID/${gameKeyU...`);
22473   if (!code) throw httpError(400, 'NO_CODE', 'wx code 缺失');
22474   const url = `https://api.weixin.qq.com/sns/jscode2session?appid=${encodeURIComponent(appid)}&se...`;
22475   const data = await httpGetJson(url);
22476   if (!data || !data.openid) {
22477     throw httpError(401, 'WX_LOGIN_FAIL', `wx code2session 失败: ${JSON.stringify(data || {})}`);
22478   }
22479   return {
22480     openid: data.openid,
22481     session_key: typeof data.session_key === 'string' ? data.session_key : '',
22482   };
22483 }
```

```
22484 async function ttCode2Openid(code) {
22485   const appid = getPlatformCredential('tt', 'APPID');
22486   const secret = getPlatformCredential('tt', 'SECRET');
22487   if (!appid || !secret) throw httpError(500, 'NO_TT_CFG', `${gameKeyUpper()}_TT_APPID/${gameKeyU...
22488   if (!code) throw httpError(400, 'NO_CODE', 'dy code 缺失');
22489   const data = await httpPostJson('https://developer.toutiao.com/api/apps/v2/jscode2session', { a...
22490   if (!data || data.err_no !== 0 || !data.data || !data.data.openid) {
22491     throw httpError(401, 'TT_LOGIN_FAIL', `dy code2session 失败: ${JSON.stringify(data || {})}`);
22492   }
22493   return data.data.openid;
22494 }
22495 async function tapCode2Session(code) {
22496   const appid = getPlatformCredential('tap', 'APPID');
22497   const secret = getPlatformCredential('tap', 'SECRET');
22498   if (!appid || !secret) throw httpError(500, 'NO_TAP_CFG', `${gameKeyUpper()}_TAP_APPID/${gameKe...
22499   if (!code) throw httpError(400, 'NO_CODE', 'tap code 缺失');
22500   const url = `https://cloud-miniapp.tapapis.cn/auth/v1/jscode2session?appid=${encodeURIComponent...
22501   const data = await httpGetJson(url);
22502   if (!data || !data.openid || data.errcode) {
22503     throw httpError(401, 'TAP_LOGIN_FAIL', `tap code2session 失败: ${JSON.stringify(data || {})}`);
22504   }
22505   return {
22506     openid: data.openid,
22507     session_key: typeof data.session_key === 'string' ? data.session_key : '',
22508   };
22509 }
22510 function httpGetJson(url) {
22511   return httpRequestJson(url, 'GET');
22512 }
22513 function httpPostJson(url, body) {
22514   return httpRequestJson(url, 'POST', body);
22515 }
22516 function httpRequestJson(url, method, body) {
22517   if (typeof fetch !== 'function') {
22518     return Promise.reject(httpError(500, 'NO_FETCH', '当前 Node 运行时不支持 fetch, 请使用 Node 18+'));
22519   }
22520   const opts = { method, headers: { 'Content-Type': 'application/json' } };
22521   if (body !== undefined) {
22522     opts.body = JSON.stringify(body);
22523   }
22524   return fetch(url, opts).then(async (res) => {
22525     const text = await res.text();
22526     try {
22527       return JSON.parse(text);
22528     } catch (_) {
22529       return { _raw: text };
22530     }
22531   });
22532 }
22533 module.exports = {
```

```
22534   handleLogin,
22535   requireUser,
22536 };
22537 // ===== FILE: cloudfunctions/cunkou-api/lib/config.js =====
22538 const DEFAULT_GAME_KEY = 'cunkou';
22539 const DEFAULT_TTL_SEC = 7 * 24 * 3600;
22540 const DEFAULT_MAX_BYTES = 256 * 1024;
22541 /** 后端 platform 字段 → 命名空间段（与客户端 gameKeyScope 一致） */
22542 function getPlatformScope(platform) {
22543   const p = String(platform || '').toLowerCase();
22544   if (p === 'dy' || p === 'tt' || p === 'douyin') return 'tt';
22545   if (p === 'tap' || p === 'taptap') return 'tap';
22546   return null;
22547 }
22548 function getGameKey() {
22549   const v = String(process.env.GAME_KEY || '').trim().toLowerCase();
22550   if (!v) return DEFAULT_GAME_KEY;
22551   if (!/^[a-z][a-z0-9_-]{0,31}$/.test(v)) {
22552     throw new Error(`非法 GAME_KEY: ${v}`);
22553   }
22554   return v;
22555 }
22556 /** 存档 / 集合 / JWT gk 使用的命名空间: 微信 cunkou, 抖音 cunkou_tt */
22557 function getScopedGameKey(platform) {
22558   const base = getGameKey();
22559   const scope = getPlatformScope(platform);
22560   return scope ? `${base}_${scope}` : base;
22561 }
22562 function scopedKeyUpper(platform) {
22563   return getScopedGameKey(platform).toUpperCase().replace(/^[A-Z0-9]/g, '_');
22564 }
22565 function gameKeyUpper() {
22566   return getGameKey().toUpperCase().replace(/^[A-Z0-9]/g, '_');
22567 }
22568 function readEnvPrefer(...keys) {
22569   for (const key of keys) {
22570     const value = process.env[key];
22571     if (value !== undefined && value !== null && String(value).length > 0) {
22572       return String(value);
22573     }
22574   }
22575   return '';
22576 }
22577 function getCollectionName(suffix, platform) {
22578   const normalizedSuffix = String(suffix || '').replace(/^_+/, '');
22579   const overrideKey = `${scopedKeyUpper(platform)}_${normalizedSuffix.toUpperCase()}_COLLECTION`;
22580   const override = process.env[overrideKey];
22581   if (override) {
22582     return String(override);
22583   }
```

```
22584   return `${getScopedGameKey(platform)}_${normalizedSuffix}`;
22585 }
22586 function getJwtSecret() {
22587   return readEnvPrefer(`${gameKeyUpper()}_JWT_SECRET`);
22588 }
22589 function getTtlSec() {
22590   const raw = readEnvPrefer(`${gameKeyUpper()}_TOKEN_TTL_SEC`);
22591   const v = Number(raw);
22592   return Number.isFinite(v) && v > 0 ? Math.floor(v) : DEFAULT_TTL_SEC;
22593 }
22594 function getMaxBytes() {
22595   const raw = readEnvPrefer(`${gameKeyUpper()}_SAVE_MAX_BYTES`);
22596   const v = Number(raw);
22597   return Number.isFinite(v) && v > 0 ? Math.floor(v) : DEFAULT_MAX_BYTES;
22598 }
22599 function getPlatformCredential(platform, field) {
22600   const upper = gameKeyUpper();
22601   const platformUpper = platform.toUpperCase();
22602   return readEnvPrefer(`${upper}_${platformUpper}_${field}`);
22603 }
22604 module.exports = {
22605   getGameKey,
22606   getPlatformScope,
22607   getScopedGameKey,
22608   scopedKeyUpper,
22609   gameKeyUpper,
22610   getCollectionName,
22611   getJwtSecret,
22612   getTtlSec,
22613   getMaxBytes,
22614   getPlatformCredential,
22615 };
22616 // ===== FILE: cloudfunctions/cunkou-api/lib/db.js =====
22617 const tcb = require('@cloudbase/node-sdk');
22618 const { getCollectionName } = require('./config');
22619 let app = null;
22620 function getApp() {
22621   if (app) return app;
22622   app = tcb.init({
22623     env: process.env.TCB_ENV || tcb.SYMBOL_CURRENT_ENV,
22624   });
22625   return app;
22626 }
22627 function getDb() {
22628   return getApp().database();
22629 }
22630 function getCollection(platform) {
22631   return getDb().collection(getCollectionName('playerData', platform));
22632 }
22633 module.exports = {
```

```
22634   getApp,
22635   getDb,
22636   getCollection,
22637 };
22638 // ===== FILE: cloudfunctions/cunkou-api/lib/http.js =====
22639 const { getGameKey } = require('./config');
22640 const CORS_HEADERS = {
22641   'Access-Control-Allow-Origin': '*',
22642   'Access-Control-Allow-Methods': 'GET,POST,OPTIONS',
22643   'Access-Control-Allow-Headers': 'Content-Type,Authorization',
22644   'Access-Control-Max-Age': '86400',
22645 };
22646 function respond(statusCode, body, extraHeaders = {}) {
22647   return {
22648     statusCode,
22649     headers: {
22650       'Content-Type': 'application/json; charset=utf-8',
22651       ...CORS_HEADERS,
22652       ...extraHeaders,
22653     },
22654     body: typeof body === 'string' ? body : JSON.stringify(body),
22655     isBase64Encoded: false,
22656   };
22657 }
22658 function preflight() {
22659   return {
22660     statusCode: 204,
22661     headers: { ...CORS_HEADERS },
22662     body: "",
22663     isBase64Encoded: false,
22664   };
22665 }
22666 function parseEvent(event) {
22667   event = event || {};
22668   if (event.httpMethod) {
22669     const method = String(event.httpMethod).toUpperCase();
22670     let rawBody = event.body || "";
22671     if (event.isBase64Encoded && rawBody) {
22672       try {
22673         rawBody = Buffer.from(rawBody, 'base64').toString('utf8');
22674       } catch (_) {}
22675     }
22676     let body = {};
22677     if (rawBody) {
22678       try {
22679         body = JSON.parse(rawBody);
22680       } catch (_) {}
22681       body = {};
22682     }
22683   }
```

```
22684     return {
22685         method,
22686         path: normalizePath(event.path || '/'),
22687         body,
22688         headers: lowercaseHeaders(event.headers || {}),
22689         query: event.queryStringParameters || {},
22690         raw: event,
22691     };
22692 }
22693 const action = String(event.action || "").replace(/^\//+/, "");
22694 return {
22695     method: 'POST',
22696     path: action ? `/${action}` : '/',
22697     body: event.body || {},
22698     headers: lowercaseHeaders(event.headers || {}),
22699     query: {},
22700     raw: event,
22701 };
22702 }
22703 function normalizePath(path) {
22704     if (!path) return '/';
22705     let p = String(path);
22706     if (!p.startsWith('/')) p = `/${p}`;
22707     const apiPrefix = `${getGameKey()}-api`;
22708     // HTTP 挂载路径可能是 cunkou-api (驼峰) , GAME_KEY 归一化后是小写 pettower
22709     p = p.replace(new RegExp(`^(?:${escapeRegExp(apiPrefix)})(?=/|$\s|i)`), "");
22710     if (p === "") p = '/';
22711     if (p.length > 1 && p.endsWith('/')) p = p.slice(0, -1);
22712     return p;
22713 }
22714 function lowercaseHeaders(headers) {
22715     const out = {};
22716     for (const key of Object.keys(headers || {})) {
22717         out[key.toLowerCase()] = headers[key];
22718     }
22719     return out;
22720 }
22721 function httpError(status, code, message) {
22722     const err = new Error(message || code);
22723     err.status = status;
22724     err.code = code;
22725     return err;
22726 }
22727 function escapeRegExp(text) {
22728     return String(text).replace(/[\.*+?^${}()|[\]\]/g, '\\$&');
22729 }
22730 module.exports = {
22731     respond,
22732     preflight,
22733     parseEvent,
```

```
22734   httpError,
22735 };
22736 // ===== FILE: cloudfunctions/cunkou-api/lib/save.js =====
22737 const { httpError } = require('./http');
22738 const { requireUser } = require('./auth');
22739 const { getCollection } = require('./db');
22740 const { getMaxBytes } = require('./config');
22741 function isCollectionNotExistError(error) {
22742   const msg = String((error && error.message) || error || "");
22743   return /collection not exist|DATABASE_COLLECTION_NOT_EXIST|ResourceNotFound|Db or Table not exi...
22744 }
22745 function emptyPullResult(userId, platform) {
22746   return {
22747     userId,
22748     platform,
22749     exists: false,
22750     schemaVersion: 0,
22751     updatedAt: 0,
22752     payload: {},
22753     payloadKeys: [],
22754   };
22755 }
22756 async function handlePull(req) {
22757   const { userId, platform } = requireUser(req);
22758   const col = getCollection(platform);
22759   let res;
22760   try {
22761     res = await col.where({ userId }).limit(1).get();
22762   } catch (error) {
22763     if (isCollectionNotExistError(error)) {
22764       return emptyPullResult(userId, platform);
22765     }
22766     throw error;
22767   }
22768   const doc = (res && Array.isArray(res.data) && res.data[0]) || null;
22769   if (!doc) {
22770     return {
22771       userId,
22772       platform,
22773       exists: false,
22774       schemaVersion: 0,
22775       updatedAt: 0,
22776       payload: {},
22777       payloadKeys: [],
22778     };
22779   }
22780   return {
22781     userId,
22782     platform,
22783     exists: true,
```

```
22784   schemaVersion: doc.schemaVersion || 0,
22785   updatedAt: doc.updatedAt || 0,
22786   payload: doc.payload || {},
22787   payloadKeys: Array.isArray(doc.payloadKeys) ? doc.payloadKeys : Object.keys(doc.payload || {}),
22788   clientFingerprint: doc.clientFingerprint || "",
22789 };
22790 }
22791 async function handlePush(req) {
22792   const { userId, platform } = requireUser(req);
22793   const body = req.body || {};
22794   const schemaVersion = Number(body.schemaVersion);
22795   const updatedAt = Number(body.updatedAt);
22796   const baseRemoteUpdatedAt = Number(body.baseRemoteUpdatedAt || 0);
22797   const payload = body.payload;
22798   const clientFingerprint = String(body.clientFingerprint || "").slice(0, 200);
22799   const force = body.force === true;
22800   if (!Number.isFinite(schemaVersion) || schemaVersion <= 0) {
22801     throw httpError(400, 'BAD_SCHEMA', 'schemaVersion 非法');
22802   }
22803   if (!Number.isFinite(updatedAt) || updatedAt <= 0) {
22804     throw httpError(400, 'BAD_UPDATED_AT', 'updatedAt 非法');
22805   }
22806   if (!Number.isFinite(baseRemoteUpdatedAt) || baseRemoteUpdatedAt < 0) {
22807     throw httpError(400, 'BAD_BASE_REMOTE_UPDATED_AT', 'baseRemoteUpdatedAt 非法');
22808   }
22809   if (!payload || typeof payload !== 'object' || Array.isArray(payload)) {
22810     throw httpError(400, 'BAD_PAYLOAD', 'payload 必须是 object');
22811   }
22812   const size = Buffer.byteLength(JSON.stringify(payload), 'utf8');
22813   const maxBytes = getMaxBytes();
22814   if (size > maxBytes) {
22815     throw httpError(413, 'PAYLOAD_TOO_LARGE', `payload 超限: ${size}B > ${maxBytes}B`);
22816   }
22817   const payloadKeys = [];
22818   for (const key of Object.keys(payload)) {
22819     if (typeof payload[key] !== 'string') {
22820       throw httpError(400, 'BAD_PAYLOAD_VALUE', `payload[${key}] 必须是字符串`);
22821     }
22822     payloadKeys.push(key);
22823   }
22824   const col = getCollection(platform);
22825   let existing = null;
22826   try {
22827     const existingRes = await col.where({ userId }).limit(1).get();
22828     existing = (existingRes && Array.isArray(existingRes.data) && existingRes.data[0]) || null;
22829   } catch (error) {
22830     if (isCollectionNotExistError(error)) {
22831       throw error;
22832     }
22833   }
```

```
22834   if (existing && !force) {
22835     const prevUpdatedAt = Number(existing.updatedAt) || 0;
22836     if (updatedAt < prevUpdatedAt || baseRemoteUpdatedAt < prevUpdatedAt) {
22837       throw Object.assign(
22838         httpError(409, 'STALE_UPDATE', `服务端已有更新版本 remote=${prevUpdatedAt} > local=${updatedAt}, b...
22839       {
22840         data: {
22841           remote: {
22842             schemaVersion: existing.schemaVersion || 0,
22843             updatedAt: prevUpdatedAt,
22844             payload: existing.payload || {},
22845             payloadKeys: Array.isArray(existing.payloadKeys)
22846               ? existing.payloadKeys
22847               : Object.keys(existing.payload || {}),
22848           },
22849         },
22850       },
22851     );
22852   }
22853 }
22854 const now = Date.now();
22855 const docData = {
22856   userId,
22857   platform,
22858   schemaVersion,
22859   updatedAt,
22860   baseRemoteUpdatedAt,
22861   clientFingerprint,
22862   payload,
22863   payloadKeys,
22864   lastWriteAt: now,
22865 };
22866 if (existing && existing._id) {
22867   await col.doc(existing._id).update(docData);
22868   return { userId, updatedAt, savedAt: now, mode: 'update', sizeBytes: size };
22869 }
22870 const addRes = await col.add(docData);
22871 return {
22872   userId,
22873   updatedAt,
22874   savedAt: now,
22875   mode: 'insert',
22876   sizeBytes: size,
22877   _id: addRes && (addRes.id || addRes._id),
22878 };
22879 }
22880 module.exports = {
22881   handlePull,
22882   handlePush,
22883 };
```